

✓ Mizan' est 100% opérationnel après la mise à jour 3.7.4.4, n'hésitez pas à reporter les bugs dans le salon associé ! [Rejoindre le Discord](#) ✕

Documentation

Comment utiliser votre espace et l'application que votre licence débloque.

Guide d'utilisation - MizanBot

Bienvenue sur **MizanBot**. Ce guide couvre l'installation, la première connexion, et l'utilisation quotidienne (comptes, mode MITM ou **mode Socket sans ouvrir Dofus**, proxys, combat, récolte, planning, équipes, **scripts Lua**...).

Besoin des **listes d'identifiants** pour tes scripts (ressources, monstres, objets) ? Tout est réuni dans l'**Annexe** tout en bas de ce guide.

1. Installation

1. Depuis ton **espace client**, télécharge **MizanBot-Setup.exe**.
2. Double-clique dessus. L'installation est **par utilisateur** : elle ne demande **aucun droit administrateur** et s'installe dans ton profil Windows. Un raccourci **MizanBot** est créé (Bureau + menu Démarrer).
3. Une fenêtre « **MizanBot — Antivirus** » te propose d'ajouter MizanBot aux **exclusions de Windows Defender** : certains antivirus signalent MizanBot **par erreur** (faux positif).
 - **Oui** → Windows te demande une **autorisation administrateur, une seule fois**.
 - **Non** → l'installation continue normalement (tu pourras toujours ajouter l'exclusion plus tard).

- Dans les deux cas, les **mise à jour** restent silencieuses (aucune autorisation redemandée).

4. MizanBot se lance automatiquement à la fin.

Windows peut afficher un avertissement SmartScreen la première fois (« éditeur inconnu ») → **Informations complémentaires** → **Exécuter quand même**.

💡 Tu utilises un **autre antivirus** que Windows Defender ? Ajoute toi-même une exclusion pour le dossier d'installation `%LOCALAPPDATA%\Mizan`.

2. Première connexion - activer ta licence

Au tout premier lancement, MizanBot te demande ta **langue** (fenêtre **Bienvenue** : Français, English ou Español ; tu pourras la changer ensuite dans les **Réglages**). Il demande ensuite ta **clé de licence** (`MZN-XXXXXX-XXXXXX-XXXXXX-XXXXXX`).

1. Colle ta clé et clique **Valider la licence**.
2. La clé se **lie à ton PC** (empreinte matérielle) au premier lancement : c'est normal, ça protège ton compte. Coche **Rester connecté** pour ne pas la ressaisir à chaque fois.
3. Ta licence donne accès à certaines **fonctionnalités** (récolte, combat, multi-comptes, adaptateurs...). Celles que ta formule n'inclut pas apparaissent **grisées** avec une explication (voir §10).

Un souci pour te connecter ? Le lien **Besoin d'aide ? Rejoindre le Discord** est en bas de la fenêtre, sous **Rester connecté**.

Changement de PC ? Réinitialise le verrou depuis ton **espace client** → **Verrou HWID** (§11), **une fois tous les 7 jours** ; au-delà, contacte le support. L'onglet **Mon compte** de l'application affiche l'empreinte de ce PC, mais ne sert pas à la réinitialiser.

Préparation des cartes. Juste après ta connexion, une fenêtre **Préparation des cartes** peut s'ouvrir : MizanBot lit **une seule fois** les cartes du jeu dans ton installation Dofus, ce qui accélère durablement tous les chargements de carte (c'est à refaire seulement après une mise à jour de Dofus). Clique **Continuer en arrière-plan** pour utiliser MizanBot pendant ce temps : une notification te prévient à la fin.

3. L'interface en un coup d'oeil

Le menu de gauche (une barre en haut avec l'**Interface façon SnowBot**, §8) donne accès aux sections principales :

Section	À quoi ça sert
Comptes	Ajouter tes comptes Dofus, leur assigner un proxy, les lancer en mode MITM (avec la fenêtre Dofus) ou en mode Socket (sans ouvrir Dofus), les Reconnecter ou les arrêter (§4)
Bots	Le poste de pilotage d'un personnage : Console, Combat, Carte, Inventaire, Banque, HDV, Lua, Stats, Planning, Paramètres, Paquets, Infos (§6)
Équipe	Regrouper tes comptes en équipes (un chef + ses membres), former le groupe Dofus, suivi et combats en commun (permission Multi , §6)
Proxys	Gérer tes proxys (SOCKS5 / HTTP, ou créés depuis un adaptateur) (§5)
Adaptateurs	Transformer une carte réseau (ex. partage 4G) en proxy local (1 IP par interface) (§5)
Réglages	Langue, thème, interface façon SnowBot, sons, effets, mods natifs et relance automatique, confidentialité, archimonstres, Discord, densité d'affichage (§8)

Mon compte

Ta licence (offre, statut, temps restant, expiration, dernière connexion), l'empreinte de ce PC, tes permissions et leurs dates d'expiration (§10)

En haut à droite de la fenêtre : l'icône **Discord** (rejoindre le Discord MizanBot) et la **cloche des notifications**. Toutes les alertes de MizanBot y restent listées (MP reçus et réponses automatiques, archimonstre repéré, bot figé, déconnexion, crash, permission activée ou retirée...), avec une pastille pour les non lues. Clique une notification pour la lire en entier ; **Tout marquer lu**, **Tout effacer** ou **Retirer** pour faire le ménage.

4. Ajouter et lancer un compte

Ajouter un compte (onglet Comptes)

Remplis l'**Identifiant** (e-mail ou pseudo Ankama), le **Mot de passe** (chiffré sur ton PC) et, si tu veux, un **Label du compte (optionnel)** (ex. Main, Mule 1 : il s'affiche en pastille sur la carte du compte). Puis, au choix :

- **Via navigateur** (recommandé) : tu te connectes à ton compte Ankama dans une fenêtre sécurisée. Choisis avant un **Proxy (optionnel)** : la connexion navigateur passe par lui (ton IP réelle n'apparaît pas), y compris un proxy **SOCKS5 avec identifiants** (typiquement un partage 4G). Sans proxy, ton IP réelle est visible pendant l'authentification Ankama. Le proxy choisi ici est aussi **assigné au compte** créé (que tu cliques **Via navigateur** ou **Ajouter le compte**).
- **Ajouter le compte** : pour un compte **déjà connecté dans le launcher Ankama** de ce PC.
- **Ajouter les comptes du Launcher Ankama** (bouton au-dessus du formulaire) : importe d'un clic **tous** les comptes déjà connectés dans le launcher officiel.

Pour corriger un compte : bouton **crayon** → **Modifier le compte** (identifiant, libellé et, pour un compte du launcher, mot de passe : laisse vide pour ne pas le changer). Bouton **corbeille** → **Supprimer** (confirmation demandée).

La carte d'un compte

- En haut : le **label**, l'identifiant, la **source** (**Ankama** = lancer, **Navigateur**) et, si un de ses persos est en connexion auto (§6), la pastille **Planning**.
- **Perso**, **Serveur** et **IP** (IP de sortie réelle) : si le perso ou l'IP est coupé « ... », survole-le pour lire la valeur complète.
- La pastille de **mode** (**MITM** ou **Socket**, voir plus bas), suivie du **statut** (Inactif, Lancement..., En jeu, Déconnecté...).
- À droite : le **type de connexion**, le **proxy**, puis les boutons **Lancer** / **Arrêter**, **Socket** / **Reconnecter**, **modifier** et **supprimer**.

Le **type de connexion** choisit où arrive le compte quand tu cliques **Lancer** : **Écran de choix du serveur**, **Choix du personnage** ou **Directement en jeu**.

Assigner un proxy (recommandé)

Choisis un **proxy** dans la liste de la carte (**Assigner un proxy** ; les proxys se créent dans l'onglet Proxys / Adaptateurs). Objectif : **1 IP par compte, zéro fuite** de ton IP réelle. « Sans proxy » = ton IP réelle est utilisée (assumé). Le proxy et le type de connexion se changent quand le compte est arrêté.

Lancer : mode MITM ou mode Socket

Chaque compte se lance de deux façons :

Bouton	Mode	Ce qui se passe
Lancer	MITM	La fenêtre Dofus s'ouvre et MizanBot la pilote : tu vois ton perso jouer, et les mods natifs (si tu les as activés, §6 et §8, et qu'ils sont inclus dans ta licence) s'appliquent
Socket	Socket	Aucune fenêtre de jeu : MizanBot se connecte directement au serveur. Beaucoup moins de CPU et de RAM, pratique pour faire tourner plusieurs comptes. Permission Mode Socket (§10)

- Le mode de ton **dernier lancement manuel** est **mémorisé** pour chaque

compte (pastille **MITM / Socket** de la carte) : **Lancer tout**, le **planning**, l'**équipe** et la **relance automatique** lanceront ce compte dans ce mode.

- **Lancer tout / Arrêter tout** (confirmation demandée). **Arrêter** coupe un compte (et annule une reconnexion en attente).
- Le multi-comptes (plusieurs comptes en simultané) nécessite la permission **Multi** (sinon un seul compte à la fois).
- Dofus installé ailleurs qu'à l'emplacement habituel ? Au premier **Lancer**, MizanBot ouvre **Localise ton Dofus.exe** : sélectionne ton `Dofus.exe`, l'emplacement est mémorisé pour la suite.
- **Code de sécurité (Bouclier Ankama)** : au premier lancement d'un compte, Ankama peut t'envoyer un **code par e-mail** → saisis-le dans la fenêtre **Code de sécurité**. MizanBot **mémorise** ensuite l'autorisation sur ce PC : plus besoin de code aux lancements suivants. (Message « Bouclier » au lancement ? voir §12.)
- Avec l'**Interface façon SnowBot** (§8), l'arbre des bots affiche ce que fait chacun : **En jeu**, **En déplacement**, **En récolte**, **En combat**, **Mort (résurrection)** pendant qu'il rejoint le Phénix (§6), **Connexion**, **Déconnecté** (voir plus bas) ou **Hors ligne**.

Le mode Socket en détail

- Sans la permission **Mode Socket**, le bouton **Socket** est grisé : survole-le pour savoir pourquoi. Le mode MITM (**Lancer**) reste toujours disponible.
- Le compte entre **directement en jeu**, sur le serveur de ton **dernier personnage joué** (avec la connexion auto du planning : sur le personnage planifié, à condition qu'il soit sur ce même serveur ; sinon la plage est ignorée, §6). Il apparaît dans l'onglet **Bots** comme un compte MITM : Console, carte, inventaire, combat, script Lua...
- Mêmes règles qu'en MITM : le proxy du compte (s'il est injoignable, le lancement est refusé : jamais de repli sur ton IP réelle), le code du Bouclier, la permission **Multi**.
- Garde Dofus **installé** sur ton PC : MizanBot y lit les cartes du jeu (§2).
- Les **mods natifs** (no-anim, mode tactique...) ont besoin de la fenêtre de jeu : aucun effet en Socket. **Enregistrer trajet** (§6) sert à noter tes déplacements

faits à la main dans Dofus : en Socket, il ne voit que ceux lancés depuis MizanBot (onglet **Carte**).

- **Coupure inattendue** (réseau, proxy 4G qui change d'IP...) : avec la **Relance automatique** (Réglages, §8, activée par défaut), le bot se reconnecte tout seul (statut « Socket : reconnexion dans ... s », au moins 15 secondes après la coupure, puis de plus en plus espacé en cas d'échec) et le script **reprend** ensuite. **Reconnecter** écourte l'attente. Après plusieurs échecs d'affilée, la reconnexion est suspendue.
- **Déconnexion imposée par le serveur** : pas de reconnexion automatique, voir ci-dessous.

Déconnecté ? Le bouton Reconnecter

Si le serveur coupe la session de ton personnage (par exemple pour **inactivité**), MizanBot le voit tout de suite : la carte du compte affiche **Déconnecté par le serveur (inactivité)** (ou **Déconnecté (...)** avec la raison), l'activité du bot passe à **Déconnecté** et une notification te prévient.

- **En MITM**, la fenêtre Dofus **reste ouverte** et le bot **reste dans l'onglet Bots**, mais son script est **arrêté** (il est gardé pour la reconnexion). Même chose si tu quittes le jeu depuis le client (retour au choix du personnage...) sans revenir en jeu dans la minute.
- **En Socket**, la session est **mise en pause** : le bot reste listé, son script est **suspendu**.
- Après une déconnexion par le serveur, MizanBot ne se reconnecte **jamais tout seul** : c'est toi qui décides (une exception en MITM si tu fermes ensuite la fenêtre Dofus, voir plus bas).

Clique **Reconnecter**, sur la carte du compte ou en haut de l'onglet **Bots** : le compte se reconnecte **tout de suite, directement en jeu sur le même personnage**, sans quitter la liste des bots, et le **script qui tournait reprend** une fois de retour en jeu (en MITM, Dofus est relancé ; en Socket, la session est reconnectée).

- **Session encore en jeu ?** Le bouton **Reconnecter** du haut de l'onglet **Bots** reste disponible (la carte du compte, elle, ne l'affiche qu'après une

déconnexion ou pendant une attente de reconnexion) : il demande confirmation (**Reconnecter ce compte ?**), car la session est coupée et un combat en cours est perdu.

- Tu reviens en jeu toi-même dans la fenêtre Dofus ? C'est possible, mais le script n'est alors **pas** repris automatiquement : relance-le avec **Lancer** (onglet Lua), sauf pendant une plage de son **Planning**, qui le relance tout seul.
- Tu as fermé la fenêtre Dofus d'un compte déconnecté **par un message du serveur** (statut **Déconnecté par le serveur (inactivité)** ou (**message système ...**)) ? Ce n'est pas compté comme un crash (pas de relance automatique) : **Reconnecter** reste proposé à côté de **Lancer** et relance le compte sur son dernier personnage, avec son script. Après une autre coupure (**connexion au serveur de jeu fermée, session de jeu quittée sans retour en jeu**), fermer la fenêtre compte comme un **crash** (voir ci-dessous).
- **Arrêter** coupe le compte et annule une reconnexion en attente.

💡 Une reconnexion relance ton script **depuis le début** (compteurs remis à zéro) : garde dans `memory:` ce qui doit survivre (§7.7).

Client fermé (crash)

Si une fenêtre Dofus se ferme de façon inattendue, la carte du compte affiche **Client fermé (crash)**. Avec la **Relance automatique** (Réglages, §8), MizanBot relance le compte tout seul, **3 fois au plus en 3 minutes** ; au-delà, la relance est suspendue (notification « trop de crashes coup sur coup ») et c'est à toi de relancer. Le script, lui, ne redémarre pas tout seul, sauf si le personnage a un **Planning** activé : pendant une plage, le planning relance son script dès le retour en jeu (et, pour un compte en **connexion auto**, c'est le planning lui-même qui relance le compte pendant sa plage, §6).

Fermer **toi-même** la fenêtre Dofus compte aussi comme une fermeture inattendue (relance automatique comprise, sauf après une déconnexion par un message du serveur, voir plus haut) : pour arrêter un compte, utilise **Arrêter**.

5. Proxys & Adaptateurs

- **Proxys** : dans **Ajouter un proxy (SOCKS5 / HTTP)**, indique un **Nom**, le **Protocole**, l'**Hôte**, le **Port** et, si besoin, **Utilisateur / Mot de passe**, puis **Ajouter** (par ex. le partage de connexion de ton téléphone, ou un proxy externe). Dans **Proxys enregistrés**, chaque proxy a ses boutons **Tester** (IP de sortie réelle + latence), **Modifier** et **Supprimer**. Les proxys **SOCKS5 / HTTP** nécessitent la permission **Proxy réseau** : sans elle, l'onglet reste visible, mais **Tester** est grisé et un compte qui utilise un de ces proxys ne se lance pas. Un proxy **créé depuis un adaptateur** dépend, lui, de la permission **Mizan Adapter** au lancement (son bouton **Tester** demande quand même **Proxy réseau**).
- **Adaptateurs** : si tu as plusieurs interfaces réseau (Wi-Fi + 4G, plusieurs cartes...), MizanBot transforme chacune en **proxy local** : pratique pour donner **une IP différente par compte** sans proxy externe. Chaque interface est listée avec son IP (**Copier l'IP**) ; active son **interrupteur** pour allumer le proxy local. Coche **Fixe** et saisis un **Port** (interrupteur éteint) pour garder toujours la même adresse (vide = port automatique). **Rafraîchir** relance la détection. Nécessite la permission **Mizan Adapter**.
- Pour donner l'IP d'un adaptateur à un compte : onglet **Proxys** → **Ou depuis un adaptateur** → choisis l'interface → **Créer depuis l'adaptateur** (le proxy apparaît avec l'étiquette **Adaptateur local**), puis assigne-le sur la carte du compte (§4). Inutile d'allumer l'interrupteur pour ça : le proxy de l'interface démarre tout seul au lancement du compte.

 Le **Tester** vérifie l'accès au web. Certains proxys (souvent 4G / résidentiels) laissent passer le web mais **bloquent le port du serveur de jeu** : la connexion Ankama marche, mais le jeu reste bloqué au chargement. MizanBot l'indique clairement dans la **Console** du bot et dans les journaux `%APPDATA%\MizanBot\logs\transport-*.log` (voir §12) : il faut alors un proxy qui autorise **tous les ports**.

6. Faire jouer un personnage (onglet Bots)

Sélectionne un personnage dans **Bots en jeu** (avec l'Interface façon SnowBot, §8 : dans l'arbre de gauche) pour ouvrir son poste de pilotage. En haut : **Copier l'identifiant**, **Reconnecter** (§4, mis en avant quand le bot est déconnecté) et **Déconnexion** (arrête le compte, comme **Arrêter**). Le panneau **État** résume le PID, l'IP de sortie, le personnage, son niveau, le serveur, la carte et les **Pods** (barre verte, ambre dès 70 %, rouge dès 90 %). Sous la liste, **Gérés par le planning** regroupe les persos en connexion auto dont le compte est hors ligne.

Les sous-onglets : **Console**, **Combat**, **Carte**, **Inventaire**, **Banque**, **HDV**, **Lua**, **Stats**, **Planning**, **Paramètres**, **Paquets** et **Infos**.

Console

Le journal du personnage (connexion, proxy, trajets...) avec, en bas, une **barre de saisie** :

- un texte normal est envoyé **en jeu** sur le canal choisi (Général, Groupe, Guilde, Alliance, Commerce, Recrutement, ou **Privé** + destinataire), 240 caractères au plus. Il faut la permission **Réponse automatique** (§10) : sans elle, rien n'est envoyé ;
- un texte qui commence par `/` est une **commande du bot**. La réponse s'affiche dans cette Console (lignes `[cmd]`, en violet) :

Commande	Effet
<code>/goto x,y</code> ou <code>/goto <id></code>	Voyage jusqu'à cette carte (à pied, zaap ou havre-sac selon le plus court)
<code>/gather</code>	Récolte la carte actuelle
<code>/fight</code>	Attaque le groupe de monstres le plus proche
<code>/cmd <instruction Lua></code>	Exécute une instruction Lua une fois (ex. <code>/cmd print(getPodsPercent())</code>) - toutes les fonctions du §7 sont disponibles
	Interrompt la commande en cours (<code>/cmd</code> , <code>/goto</code> ,

<code>/abortCmd</code>	<code>/gather</code> , <code>/fight</code>)
<code>/cellid</code>	Affiche la cellule où se trouve ton personnage
<code>/mapid</code>	Affiche l'id et les coordonnées de la carte actuelle
<code>/id NOM</code>	Affiche l'id d'un joueur ou d'un PNJ présent sur ta carte (pour un PNJ, aussi son id de modèle : <code>Nom : -12 (modèle 522)</code> , celui à utiliser dans les scripts)
<code>/joinBoss</code>	<i>(chef d'équipe)</i> Envoie tous les membres de l'équipe sur ta carte
<code>/clear</code>	Vide la Console

Les commandes **n'arrêtent pas** le script en cours et respectent tes droits (ex. `/fight` exige **Combat auto**, `/gather` exige **Récolte auto**).

En haut à droite de la Console : **Copier** (tout, ou seulement les lignes sélectionnées : clic, Ctrl+clic, Maj+clic, puis Ctrl+C) et **Vider**.

Combat

Deux vues : **Comportement** et **Sorts**.

Comportement

- **Jouer automatiquement mes tours de combat** (décoché par défaut) : active le combat auto. Coche-le pour que le bot joue tes tours, y compris dans les combats lancés par ton script ou par `/fight` ; sinon le combat démarre mais personne ne joue à ta place (sauf `onTurn()` , voir plus bas).
- **Résurrection auto (Phénix)** (activée par défaut) : si ton personnage meurt, il libère son âme, **voyage jusqu'au Phénix** indiqué par le jeu, **ressuscite**, puis le script reprend. Ça marche **avec ou sans script** en cours (combat seul, membre d'équipe...) et ne demande pas le droit **Combat auto** (utiliser le Phénix demande seulement **Récolte auto**). Pendant ce temps, l'activité du bot indique **Mort (résurrection)**.

- **Pré-combat auto** : se placer (selon le style) et se mettre prêt, et Activer un **challenge** au placement (le bonus XP/Butin reste celui que tu as choisi en dernier dans le jeu). En mode MITM, désactive l'option du jeu qui te met prêt automatiquement : c'est le bot qui s'en charge.
- **Style d'IA** : *Agressif* (se rapproche puis lance ses sorts), *Fuyard (kite)* (lance puis s'éloigne), *Passif* (lance seulement si déjà à portée).
- **Cible** : *Le plus proche*, *PV les plus bas* ou *Le plus loin*. **Vitesse** : *Instantané*, *Rapide (0,5–1 s)* ou *Lent (1–3 s)*.
- **Achever en priorité les cibles tuables ce tour** (activé par défaut) : un ennemi que ta rotation peut tuer ce tour passe en première cible.
- **Lancers max / tour** : nombre maximum de sorts lancés par tour, tous sorts confondus.
- **Distance kite min** et **Distance kite max (0 = ∞)** : en style Fuyard, le bot reste entre ces deux distances de l'ennemi le plus proche.
- **Auto level-up** : Dépenser mes points de caractéristique à chaque montée de niveau dans la **Caractéristique cible** (Force, Intelligence, Chance, Agilité, Vitalité ou Sagesse). Droit **Combat auto**.
- **Confort client (mods natifs)** : *No-anim* (animations de combat instantanées), *Collapse* (attentes de combat instantanées), *Déplacements instantanés* (téléportation en combat, expérimental) et *Mode tactique* (overlay des cases marchables / obstacles). Appliqués à chaud (relance un client déjà ouvert pour la première injection). Ils agissent sur la fenêtre Dofus : il faut **Injecter les mods natifs** activé dans les Réglages (§8, activé par défaut), et ils sont sans effet en mode Socket.
- **Simuler le tour** : place ton perso et des ennemis sur une grille, règle PA / PM / portée, puis **Simuler** pour voir ta rotation jouée pas à pas (option **Rejouer jusqu'à la fin du combat**). C'est un outil pour comprendre et régler ta rotation (dégâts moyens, sans résistances), pas un rejeu exact.
- **IA scriptée** : si ton script Lua définit une fonction `onTurn()`, c'est lui qui joue tes tours (voir §7.12) ; sinon, c'est ce module qui s'en charge.

Sorts : + **Ajouter un sort...**, puis règle l'ordre (glisse la pastille de priorité ou utilise les flèches : le sort le plus haut passe en premier). Le bouton **Configurer** d'un sort ouvre trois onglets :

- **Essentiel** : priorité, lancers $\times$ / tour, **Relance** (tours minimum entre deux lancers), et les règles rapides : **Lancer directement sur moi**, **Buff** : lancer même sans ennemi à portée, **Invocation** (placer sur une cellule libre), **Cibler un allié** (soin/buff) ;
- **Placement & zones** : **Distance mini** et **Portée max** de tir, **Se déplacer pour lancer ce sort** ;
- **Conditions avancées** : PV de la cible, PV du personnage, **PA min**, **Tour $\geq$** , **Ennemis $\geq$** , **En zone $\geq$** (le sort n'est lancé que si toutes ses conditions sont remplies).

Récolte (pas d'onglet dédié)

La récolte se pilote par ton **script Lua** (§7 : `gather = true`, `gatherAll()`, `ELEMENTS_TO_GATHER`), par la commande `/gather` (Console) ou par un **clic droit** sur une ressource dans l'onglet **Carte**. Elle exige la permission **Récolte auto**. Sans filtre de ressources, le bot ne récolte que les **vraies ressources** : il ne clique jamais sur une porte, un zaap ou un élément « Utiliser ».

Carte

La carte tactique de ta position : sol marchable, obstacles, et chaque acteur sur sa cellule (**Légende** : Vous, Joueurs, Monstres, PNJ, Interactif / ressource, Archimonstre).

- **Clic gauche** sur une case : ton personnage y marche.
- **Clic droit** : récolter / utiliser l'interactif de la case (ex. « Couper — Frêne », grisé s'il est indisponible) ou **Attaquer** le groupe de monstres (selon tes droits).
- **Changer de carte** : boutons **Haut**, **Gauche**, **Droite**, **Bas**.
- **Afficher les cellules de transition** (les sorties de la carte) et **Afficher les numéros de cellule** : pratique pour trouver une cellule à mettre dans un script (`cell`, `door`, `goToCellId` ...).
- **Informations** : **Map id**, **Position** et **Cellule** (celle sous ta souris) ; dessous, **Acteur survolé** (son id et, pour un PNJ, son **modèle** : l'id à utiliser dans les scripts).
- **Archimonstres** : un groupe qui contient un archimonstre est mis en

évidence, une notification te prévient (« Archimonstre ... repéré ») et le panneau **Archimonstres récents** en garde la liste. Le MP Discord se règle dans les **Réglages** (§8).

Inventaire / Banque

- **Inventaire** : le contenu du sac, avec pour chaque objet sa quantité et son **gid** (l'id à utiliser dans les scripts). **Rechercher (nom, gid)...**, filtre par catégorie et tri (**défaut, nom (A→Z), quantité, catégorie**). Sac vide au lancement ? Reconnecte le personnage pour le charger.
- **Banque** : tes **kamas** en banque et le contenu du coffre (mêmes recherche, catégories et tri), chargé dès que tu ouvres ta banque (en jeu ou par un dépôt du script).
- **Type de dépôt** (réglage **commun à tous tes personnages**) : ce que déposent la banque automatique (`npcBank` , §7.5) et les coffres à code : **Déposer tout l'inventaire** (par défaut), **Déposer les objets déjà présents** (seulement ceux que ton coffre contient déjà) ou **Déposer les objets visibles** (les objets de ton sac que MizanBot voit, hors équipement porté).

HDV (permission Hôtel de Vente)

Deux vues : **HDV** et **Statistiques**.

- **Mes ventes en cours** : tes lots avec le **Prix marché**, **Mon prix** et un **État** (au meilleur prix, plus cher que le marché, prix inconnu). Pour chaque lot, **Mon prix** se modifie directement, et les icônes de la ligne (survole-les) : la case **Ignorer (exclure du reprice auto)**, – (passer sous le prix marché, sauf si tu es déjà le moins cher), le crayon **Repricer** (applique **Mon prix**) et la corbeille **Retirer**. Plus **Sous-coter de X kamas**, **Tout sous-coter** et **Rafraîchir**. Ouvre le HDV **en mode vente** en jeu pour charger tes lots ; sans fenêtre de jeu (mode Socket), c'est **Démarrer le repricing auto** (ci-dessous) qui s'y rend et l'ouvre pour toi.
- **Contrôle** : choisis l'**Hôtel de Vente cible**, règle le **Repricing automatique** (**Rafraîchissement (ms)** et **Délai entre ventes (ms)**, tirés au hasard entre min et max, et **Plancher : % sous le marché**), puis **Démarrer le repricing auto** : le bot voyage jusqu'au HDV, l'ouvre et reprice tes lots en boucle (**Annuler**

pendant le voyage). Le **Journal d'activité** trace chaque action.

- **Bilan** : ventes, **Taxes payées** et **Plafond de taxe (0 = illimité)** ; au-delà du plafond, le repricing s'arrête et la mise en vente est bloquée. Chaque reprise coûte une taxe en jeu.
- **Mettre en vente (sac + banque)** : cherche un objet, regarde son **Prix actuel**, choisis la **Quantité** et le **Prix du lot**, puis **Mettre en vente** (confirmation demandée).
- **Statistiques** : sur **24 h**, **7 jours**, **30 jours** ou **1 an** : **Profit net**, **Lots vendus**, courbe du **Profit net cumulé**, **Profit net par objet**, **Journal détaillé** et **Exporter CSV**.

Lua

Le coeur de l'automatisation : tu écris un petit **script** qui décrit ce que ton personnage fait en boucle (récolter une zone, enchaîner des combats, déposer en banque...). Simple à démarrer et très puissant : tout est expliqué à la §7.

Enregistrer trajet : crée un script **sans rien écrire**, en jouant à la main (compte lancé avec **Lancer**, puisque tu te déplaces dans la fenêtre Dofus).

1. Sois en jeu sur une carte, puis clique **Enregistrer trajet** (en haut de l'onglet Lua). Le bouton devient **Arrêter l'enregistrement** (point rouge).
2. Déplace-toi **à la main** en jeu, carte par carte (à pied, par une porte, un PNJ, un zaap...). À chaque nouvelle carte, une fenêtre **Nouvelle carte** te demande : **Récolter sur cette carte** et/ou **Combattre sur cette carte** → **Confirmer** (ou **Passer**).
3. Reviens sur ta carte de départ pour **boucler** le trajet, puis clique **Arrêter l'enregistrement** : le script est **placé dans l'éditeur** et MizanBot te propose de **l'enregistrer** en `.lua`.
4. Complète si besoin `ELEMENTS_TO_GATHER` en tête du script (vide = toutes les ressources, §7.4), puis **Lancer**.

Une carte = une étape (une seule sortie par carte). Tu peux retoucher le script généré comme n'importe quel autre (§7.5). Un **Reconnecter** (§4) interrompt l'enregistrement : arrête-le puis relance-le.

Stats

Le bilan du personnage sur **24 h, 7 jours, 30 jours, 1 an** ou un jour précis (**Choisir un jour précis**) : récoltes, combats, morts, temps moyen par combat, temps de botting, kamas et XP gagnés, ressources, pods récoltés par heure, passages en banque, dernières actions, ce que tu as récolté sur la période et la **Heatmap du trajet** (passages, récoltes et temps moyen par carte). **Exporter CSV** pour tes propres tableaux.

Planning

Définis une **semaine type** de plages horaires : MizanBot **démarre et arrête** le script tout seul sur ces créneaux, pour un rythme plus **humain** (évite de tourner 24h/24 de façon suspecte). Chaque personnage a son propre planning.

- **Planning activé** met le planning en marche ; son état (**Plage active, En attente, Désactivé**) s'affiche à côté.
- **Humanisation ($\pm$ min)** décale légèrement les heures de début et de fin, pour ne jamais démarrer pile à l'heure ronde.
- **Bibliothèque de scripts** : **Charger un .lua** (un ou plusieurs fichiers ; le nom du fichier devient le nom du script) ou **Ajouter le script courant** (fige le script de l'éditeur Lua sous le **Nom du script** saisi).
- **Ajouter une plage** : donne-lui un nom (ex. « Pêcheur »), ses **jours** (Lu à Di), ses horaires et son **Script de la plage**, choisi dans la bibliothèque. Une plage sans script lance le script du personnage (celui de son onglet Lua). Une plage qui en chevauche une autre est signalée en rouge.
- **Figer le script courant** enregistre le script de l'éditeur Lua comme script à lancer par le planning.

Si le script s'arrête parce qu'il est **bloqué** (trajet impossible, voir §7.9), le planning se met **en pause jusqu'à la fin de la plage** en cours, avec une notification, au lieu de le relancer en boucle. Il reprend normalement à la plage suivante (ou dès que tu cliques **Lancer** toi-même).

Connexion auto (interrupteur à côté de « Planning activé ») : un vrai planning. MizanBot **connecte le compte** au début de chaque plage, directement en jeu, et lance le script. **En dehors des plages**, le compte est **déconnecté** (après le

combat en cours), même si tu l'as lancé à la main.

- **Avant chaque déconnexion**, une fenêtre avec un **compte à rebours de 10 secondes** s'ouvre : **Garder en ligne** laisse le compte connecté jusqu'à sa prochaine plage, **Déconnecter maintenant** coupe tout de suite. Sans réponse, il est déconnecté à la fin du décompte.
- **Pas de risque de te bloquer dehors** : tant que le planning n'a aucune plage, rien n'est jamais déconnecté.
- Pour démarrer, lance le compte **une fois à la main** et active la connexion auto dans l'onglet Planning du perso : c'est ce qui relie le perso à son compte.
- Le compte se connecte dans le **mode** retenu pour lui (pastille **MITM** / **Socket** de sa carte, §4). En **Socket**, le personnage planifié est choisi **directement**, à condition qu'il soit sur le serveur de ton **dernier perso joué** sur ce compte (§4). En **MITM**, le compte se connecte sur son **dernier perso joué** : joue une fois le bon perso sur ce compte. Si un autre perso arrive en jeu, il est déconnecté, tu es prévenu et la plage est ignorée.
- Si une autre plage commence dans les 10 minutes (plus deux fois l'humanisation), le compte reste connecté ; la console l'indique.
- **Arrêter** un compte pendant une plage : il n'est pas relancé avant la fin de cette plage.
- Un perso géré par le planning **reste visible dans l'onglet Bots** quand son compte est hors ligne : ligne grisée « Hors-ligne · géré par le planning », avec l'heure de sa prochaine connexion. Clique dessus pour **modifier son planning** ou le **connecter maintenant** (il ne sera alors pas déconnecté avant sa prochaine plage). Sur la page Comptes, un badge « Planning » marque les comptes concernés.
- En cas d'échec ou de crash, MizanBot réessaie (3 tentatives au maximum par plage). Si le compte n'arrive pas à se connecter **2 plages de suite**, la connexion auto est suspendue jusqu'à ce que tu le lances à la main.
- Un compte qui demande un **code du Bouclier** ne peut pas être connecté sans toi : lance-le une fois à la main (l'appareil est ensuite mémorisé).
- Fermer la fenêtre Dofus d'un compte connecté par le planning compte comme un crash : il est relancé pendant la plage. Pour l'arrêter, utilise **Arrêter** dans MizanBot. Exception : si la carte du compte affichait

Déconnecté par le serveur (inactivité) ou (message système ...) (§4), fermer la fenêtre vaut un **Arrêter**.

- Un compte **Déconnecté** (§4) n'est pas reconnecté tout seul par le planning : clique **Reconnecter** (§4), ça ne compte pas comme un arrêt et la plage continue normalement. En mode Socket, une coupure réseau est reprise par la **Relance automatique** (§8, activée par défaut), sans que le planning n'intervienne (si tu l'as désactivée, c'est le planning qui relance le compte pendant sa plage).
- Les scripts ajoutés au planning **depuis un fichier** `.lua` sont **rechargés depuis le fichier** au début de chaque plage : modifie ton trajet sur le disque, il est pris en compte à la plage suivante.

Paramètres

Les réglages **propres à chaque personnage** :

- **Déplacement** : **Vitesse Rapide** (défaut) ou **Legit** (plus humain) ; en Legit, règle le **Délai après changement de carte (ms)** — min / max (1500 à 3000 par défaut).
- **Messagerie privée** (permission **Réponse automatique**) : coche **Répondre automatiquement aux messages privés** et choisis le **Mode** (tableau ci-dessous). **Délai avant réponse (ms)** — min / max et **Cooldown par joueur (s)** gardent un rythme humain. Chaque MP reçu (et chaque réponse automatique ou envoyée depuis Discord / ton espace client) apparaît dans la cloche des notifications.
- **Relais Discord** : **Activer le relais Discord** pour recevoir tes MP en jeu sur Discord et y répondre (ton Discord doit être lié depuis ton espace client, §11). **Fenêtre réponse manuelle (s)** : le temps laissé pour répondre toi-même ; avec **Réponse auto si pas de réponse manuelle**, la réponse automatique part ensuite.

Mode	Effet
Désactivé	Aucune réponse automatique
Mots-	Répond selon tes Règles (mot-clé → réponse) (<i>Contient, Égal à</i>

<i>clés</i>	ou <i>Commence par</i>) : Ajouter une règle / Retirer
<i>IA (LLM)</i>	Répond avec une IA : Modèle , Endpoint (compatible OpenAI ; vide = OpenAI) , Clé API (chiffrée sur ton PC, Effacer pour la retirer) et Prompt système
<i>Mots-clés puis IA</i>	Essaie tes règles, puis l'IA si aucune ne correspond

Anti-blocage : repère un bot **vivant mais figé**, c'est-à-dire un script qui tourne mais n'avance plus (boucle qui n'agit pas, coincé dans un dialogue, bloqué après une carte). Le combat, la récolte, le déplacement et la mort ne sont **jamais** concernés.

Réglage	Par défaut	Effet
Surveiller les blocages (déblocage doux : fermeture d'un dialogue)	Activé	Après le délai sans aucune action, le bot ferme un éventuel dialogue resté ouvert
Redémarrer le script si toujours bloqué (plus intrusif)	Désactivé	Toujours figé au double du délai : le script est relancé
Délai avant blocage (s)	180	De 30 à 3600 s. Augmente-le si tes scripts font de longues pauses (<code>delay</code>)

Toujours figé au **triple** du délai : MizanBot **abandonne** et t'affiche une notification « **bot figé** » (intervention manuelle conseillée). L'anti-blocage ne **reconnecte** jamais le compte.

Paquets / Infos

Paquets : l'inspecteur des messages du jeu (surtout utile au support et aux scripts avancés). **Infos** : classe, sexe, PV, PA, PM et PO du personnage.

L'onglet Équipe (multi-comptes)

Regroupe tes comptes en équipes (permission **Multi**). Il faut au moins **2 comptes en jeu** pour composer une équipe.

1. **Nouvelle équipe**, puis **Ajouter un membre...** pour chaque perso, et **Définir chef** sur l'un d'eux (badge **Chef**). La pastille de couleur (**Changer la couleur**) repère l'équipe dans la liste des bots une fois le groupe formé. Les membres restent enregistrés, qu'ils soient en ligne ou non (**Retirer du groupe** pour en enlever un).
 2. **Connecter en jeu** lance directement en jeu chaque membre pas encore connecté, chacun avec **son** proxy, dans **son** mode (MITM ou Socket, §4) et sur son **dernier perso joué** (en Socket : sur le serveur de son dernier perso joué, §4) ; le chef invite chaque membre dès son arrivée (un autre perso que celui de l'équipe n'est pas invité). Un perso jamais vu en jeu sur ce PC doit d'abord être lancé une fois depuis l'onglet **Comptes**.
 3. **Former le groupe** : le chef invite les membres en jeu dans un vrai groupe Dofus ; ils acceptent tout seuls. Les membres doivent être sur le **même serveur** que le chef.
 4. **Démarrer le groupe** lance le **script du chef** (il doit en avoir un dans son onglet Lua), les membres le suivent. **Arrêter le groupe** coupe les scripts et le suivi de tout le monde. **Dissoudre l'équipe** arrête le mode équipe (suivi, combats communs, verrouillage des membres) ; le groupe Dofus, lui, n'est pas quitté automatiquement.
- **Suivi automatique du chef** : les membres collent au chef (carte et cellule) et l'attendent avant de changer de zone ; le **Délai inter-bots aléatoire (min → max, ms)** échelonne leurs actions.
 - **Rejoindre les combats du chef** : les membres rejoignent chaque combat, celui de **n'importe quel** coéquipier ; le chef les attend avant d'engager et avant de choisir le challenge.
 - Une fois le groupe formé, un membre est **piloté par son équipe** : son propre script est arrêté, son script et son planning sont verrouillés, seul son onglet **Combat** reste réglable.
 - La **Console Équipe (tous les bots du groupe)** réunit le journal de tous les

membres. Dans la Console d'un bot, `/joinBoss` (chef) envoie tous les membres sur sa carte.

7. Écrire ses propres scripts Lua

MizanBot embarque un **moteur de script Lua** : au lieu de cliquer, tu **écris** ce que ton personnage doit faire, et le bot le répète en boucle. Pas besoin d'être développeur : on part d'un exemple et on l'adapte.

En deux mots : un script définit une fonction `move()`. À chaque tour de boucle, MizanBot appelle `move()` et exécute ce qu'elle décrit. C'est tout.

 Cette partie est une **prise en main**. La liste complète des fonctions, avec tous leurs détails, est dans la **référence de l'API** (espace client → **Documentation**).

7.1 L'onglet Lua : charger, lancer, suivre

Dans **Bots** → **Lua** :

- **L'éditeur** : clique l'en-tête **Script Lua** pour le déplier (il est replié par défaut). Chaque personnage a **son propre script**, mémorisé ; un nouveau perso démarre avec un **script d'exemple** qui te montre l'API.
- **Charger** : ouvre un fichier `.lua` depuis ton PC (son chemin s'affiche dans l'en-tête de l'éditeur).
- **Enregistrer trajet** : génère un script en enregistrant tes déplacements manuels (§6).
- **Lancer / Arrêter** : un seul bouton. Un **point vert** clignote tant que le script tourne ; le bouton repasse sur **Lancer** tout seul si le script s'arrête (fin, trajet bloqué...).
- **La Console du script** (dessous) journalise **chaque action** en temps réel, horodatée et colorée, avec une **catégorie** entre crochets (`[script]`, `[récolte]`, `[combat]`, `[déplacement]`, `[carte]`, `[trajet]`, `[banque]`, `[inventaire]` ...). Tu peux **filtrer** (**Tout** / **Normal** / **Avertissements** / **Erreurs**),

Copier (tout, ou seulement les lignes sélectionnées : clic, Ctrl+clic, Maj+clic, puis Ctrl+C) ou **Vider**.

- Le journal est aussi **enregistré sur disque** : `%APPDATA%\MizanBot\lua\<NomDuPersonnage>.log`.

💡 Le **Planning** (§6) peut lancer et arrêter tes scripts automatiquement sur des créneaux horaires : chaque plage lance le script choisi dans sa **Bibliothèque de scripts**, ou à défaut le script de ce personnage.

💡 Pour tester une fonction sans lancer de script, tape-la dans la **Console** avec `/cmd` (§6), ex. `/cmd print(getLifePointsPercent())`.

7.2 Deux façons d'écrire un script

A. Le mode déclaratif (recommandé pour débiter, idéal pour le farm de zone).

Tu décris un **itinéraire** : `move()` renvoie une **liste d'étapes**, une par carte. Pour chaque carte tu dis « ici je récolte, puis je pars à droite ». MizanBot suit l'itinéraire, récolte / combat selon tes drapeaux, et recommence en boucle.

B. Le mode impératif (avancé, logique sur mesure). `move()` **exécute directement** des ordres (`gatherAll()`, `fight()`, `goToCellId()`, `changeMap()`, `travelTo()` ...). Tu ajoutes des conditions (`if`), des compteurs, etc. Plus de liberté, un peu plus technique.

Tu peux mélanger les deux : commence en déclaratif, passe à l'impératif quand tu veux plus de contrôle.

7.3 Ton premier script (déclaratif), commenté

Ce script récolte du **Blé** sur une petite boucle de 4 cartes, puis recommence :

```
-- Ce qu'on récolte (par "ElementTypeId" - voir l'Annexe). 38 = Blé.
ELEMENTS_TO_GATHER = { 38 }

-- Quand le sac est plein à 90 %, on passe en mode banque (si bank() e
MAX_PODS = 90
```

```
-- move() renvoie l'itinéraire : une ligne = une carte.
function move()
  return {
    -- map = coordonnées de la carte | path = par où sortir ensuite |
    { map = "4,-22", path = "right", gather = true, fight = false },
    { map = "5,-22", path = "bottom", gather = true, fight = false },
    { map = "5,-21", path = "left", gather = true, fight = false },
    { map = "4,-21", path = "top", gather = true, fight = false },
  }
end
```

Comment le lire : sur la carte `4,-22` → récolte le blé, puis sors par la **droite** ; arrivé sur `5,-22` → récolte, sors par le **bas** ; etc. Après la dernière carte, la boucle repart de la première.

Pour construire **ta** boucle : place-toi en jeu, regarde tes **coordonnées** en haut à droite de Dofus, et liste tes cartes dans l'ordre avec la direction de sortie de chacune.

💡 Encore plus simple : **Enregistrer trajet** (onglet Lua, §6) écrit ce `move()` pour toi pendant que tu fais le parcours à la main.

7.4 Les réglages en tête de script

Ces **variables globales** se mettent en haut du script (toutes optionnelles) :

Variable	Rôle	Exemple
<code>ELEMENTS_TO_GATHER</code>	Liste des types de ressources à récolter (vide/absent = toutes les vraies ressources , jamais les portes, zaaps ou « Utiliser »)	<code>{ 38, 39 }</code>

MAX_PODS	Seuil de remplissage (%) qui déclenche le passage en banque (95 par défaut)	MAX_PODS = 90
MIN_MONSTERS	Taille minimale d'un groupe pour l'attaquer (1 par défaut)	MIN_MONSTERS = 1
MAX_MONSTERS	Taille maximale d'un groupe pour l'attaquer (aucune limite si absent)	MAX_MONSTERS = 4
FORBIDDEN_MONSTERS	Groupes à ne jamais attaquer (par nom ou par id)	{ "Tofu", 123 }
FORCE_MONSTERS	N'attaquer que les groupes contenant l'un de ceux-ci	{ "Bouftou" }
MONSTERS_AMOUNT	Pour un monstre donné (genericId) : nombre min / max accepté dans un groupe. Un groupe qui en contient trop (ou pas assez) est ignoré ; un groupe sans ce monstre n'est pas concerné	{ { 36, 0, 2 } } (au plus 2 Bouftous)
OPEN_BAGS	Sac plein → ouvre d'abord tous les sacs de ressources, avant	OPEN_BAGS = true

	d'aller en banque	
AUTO_DELETE	Sac plein → détruit ces objets (dans le sac seulement, jamais un objet équipé)	{ 311, 1984 }
MIN_LIFE_PERCENT	Avant d'attaquer, se régénère si les PV sont sous ce % (0 = désactivé)	MIN_LIFE_PERCENT = 50
REGEN_UP_TO_PERCENT	Jusqu'où remonter les PV pendant la régénération (100 par défaut)	REGEN_UP_TO_PERCENT = 90
REGEN_FOOD	Consommables (gid, voir l'Annexe) à utiliser d'abord pour se soigner, dans l'ordre	{ gid1, gid2 }
REGEN_SIT / REGEN_MAX_WAIT	S'asseoir pendant la régénération (oui par défaut) / attente max en secondes (600)	REGEN_SIT = false
FIGHT_LOCK_SPEC / FIGHT_LOCK_JOIN / FIGHT_LOCK_PARTY	Au début de chaque combat (agressions comprises ; en équipe, c'est le chef qui les applique) : bloquer les spectateurs / fermer aux arrivants (après	FIGHT_LOCK_SPEC = true

	l'arrivée de ton équipe) / réserver au groupe	
FIGHT_KICK	Au placement, expulser après ce délai (ms) tout joueur qui n'est pas de ton équipe MizanBot. Absent = personne n'est expulsé	FIGHT_KICK = 3000
IGNORE_REQUEST_EXCHANGE / IGNORE_REQUEST_DUEL	Refuser automatiquement les demandes d'échange / de défi d'autres joueurs (jamais celles de tes propres bots)	IGNORE_REQUEST_DUEL = true
IGNORE_REQUEST_GUILD	Refuser automatiquement les invitations de guildes d'inconnus (celles de tes propres persos te sont laissées)	IGNORE_REQUEST_GUILD = true
MODERATOR_STOP / MODERATOR_WEBHOOK	Message d'un modérateur → arrêter le script (et mettre le planning en pause) / le relayer sur un webhook Discord	MODERATOR_STOP = true
	Temps max (ms) laissé à ton	

TURN_TIME_LIMIT	onTurn() (et à onPlacement()) : 20000 par défaut, entre 3000 et 25000 (§7.12)	TURN_TIME_LIMIT = 15000
PLANNING	Heures (0 à 23) où le bot doit être hors ligne , comme sous SnowBot : il se déconnecte, se reconnecte tout seul à la première heure libre et reprend le script. Indépendant de l'onglet Planning (§6)	PLANNING = { 3, 4, 5 }
SNOWBOT_TIMEOUT	Délai de l' Anti- blocage (§6) pour ce script, en tours SnowBot de 6 s (60 s minimum)	SNOWBOT_TIMEOUT = 50
GATHER	Autre nom (SnowBot) de ELEMENTS_TO_GATHER	GATHER = { 38 }

7.5 Les champs d'une étape d'itinéraire

Chaque { ... } renvoyé par move() (ou bank()) accepte les champs suivants, joués dans l'ordre du tableau :

Champ	Type	Effet
map	"x,y" ou id de carte (nombre ou texte), ou "havenbag"	La carte à laquelle s'ap ("havenbag" = une éta

		/ <code>path</code>
<code>custom</code>	<code>function</code>	Bloc Lua exécuté sur ce (verbes impératifs disponibles de carte / lance un commandement s'arrête là. S'il renvoie une liste d'étapes , ce mini-trajet
<code>lockedCustom</code>	<code>function</code>	Comme <code>custom</code> , mais un membre de l'équipe à la
<code>door</code>	cellule ou <code>"a b c"</code>	Franchir l'interactif/portale cette cellule (plusieurs : au hasard)
<code>exitCell</code>	<code>number</code>	Quitter la carte par cette bord précise
<code>path</code> (ou <code>changeMap</code> , <code>paths</code>)	voir ci-dessous	Par où quitter la carte ou (<code>changeMap</code> = même chose prioritaire ; <code>paths</code> = no

Coffre ou maison à code : un mauvais code arrête le script ; un coffre occupé est retenté 5 s plus tard.

Les valeurs possibles de `path` / `changeMap` :

Forme	Exemple	Effet
Direction	<code>"right"</code> , <code>"haut"</code> , <code>"top(12)"</code>	Change de carte par ce bord (synonymes FR acceptés) ; <code>(12)</code> = en sortant par cette cellule
Au hasard	<code>"left right"</code> ou <code>{ "left", "right" }</code>	Une des sorties, tirée au hasard (avec la liste <code>{ }</code> , en évitant si possible de revenir sur la carte

		d'où tu viens)
Au hasard pondéré	<code>{ { direction = "left", weight = 50 }, { path = "right", weight = 30 } }</code>	Tirage selon les poids
Coordonnée	<code>"4,-18"</code>	Adjacente : simple changement de carte ; lointaine : le bot voyage jusque-là
Id de carte	<code>88085761</code> ou <code>"88085761"</code>	Voyage multi-cartes jusqu'à cette carte (zaap / havre-sac si utile)
Plusieurs ids	<code>{ 154133, 154134 }</code>	Une des cartes au hasard, puis voyage
Zaap / zaapi	<code>"zaap(88085761)"</code> , <code>"zaapi(<id>)"</code>	Prend le zaap (ou le zaapi) de la carte vers cette destination (sinon, voyage)
Havre-sac	<code>"havenbag"</code>	Entre dans le havre-sac (ou en ressort)

Les préfixes des scripts écrits pour d'autres bots (`goto:`, `travel:`, `usezaap:` ...) sont tolérés.

Plusieurs étapes pour la même carte (fréquent dans les scripts SnowBot) : elles sont **fusionnées**, et pour chaque champ c'est la **dernière** étape écrite qui gagne. Si une étape vise l'**id exact** de la carte, les étapes en coordonnées `"x,y"` sont ignorées sur cette carte (un intérieur a la même coordonnée que l'extérieur).

7.6 La banque automatique

Ajoute une fonction `bank()` qui renvoie l'itinéraire jusqu'à ta banque, avec une

étape `npcBank = true` :

```
function bank()
  return {
    { map = "192415750", npcBank = true } -- Banque d'Astrub (par id
  }
end
```

Dès que le remplissage atteint `MAX_PODS`, MizanBot **bascule tout seul** sur `bank()`, va déposer, puis reprend la récolte quand le sac est de nouveau libre. Tu peux mettre l'**id** exact de la carte de banque (le bot voyagera jusqu'à elle tout seul). Ce qui est déposé suit le **Type de dépôt** de l'onglet **Banque** (§6).

Si tu as mis `OPEN_BAGS` et/ou `AUTO_DELETE` (§7.4), ils sont joués **avant** : le bot ne part en banque que si le sac est **encore** plein ensuite. Sans `bank()`, la console affiche un avertissement « pods pleins » et `move()` continue. Tu préfères un coffre à code ? Utilise `lockedStorage` (§7.5).

💡 Déclarer `function bank()` ne te prive pas du module `bank:` (§7.7) : dans `move()`, `bank()` et les hooks, `bank:open()`, `bank:withdraw(gid, qty)`... fonctionnent normalement. Seule exception : en tête de fichier, juste après la ligne `function bank()`.

7.7 Les fonctions disponibles (référence)

Tu peux appeler ces fonctions n'importe où dans ton script (surtout utile en mode **impératif**). Les fonctions de déplacement / récolte / combat sont **bloquantes** : elles attendent la confirmation du serveur avant de rendre la main, et renvoient `true` / `false` selon la réussite. Voici les plus utiles ; la liste complète est dans la **référence de l'API** (espace client → Documentation).

Journal & contrôle

Fonction	Effet
<code>print(txt)</code>	Ligne d'info dans la console

<pre>printSuccess(txt) / printWarning(txt) / printError(txt)</pre>	Ligne verte / jaune / rouge
<pre>printMessage(txt, "info" "success" "warning" "danger")</pre>	Ligne colorée par niveau
<pre>printColor(txt, "#RRGGBB")</pre>	Couleur libre (hex). <code>...</code> <code></code> dans le texte = gras
<pre>delay(ms)</pre>	Pause (millisecondes), légèrement variée ($\pm 15\%$) pour paraître plus humaine, interrompue proprement si tu cliques Arrêter
<pre>notify(txt)</pre>	Notification dans MizanBot (avec un son)
<pre>stopScript()</pre>	Arrête le script

Lire l'état du personnage

Fonction	Renvoie
<code>getMapId()</code>	Id de la carte courante
<code>getCurrentPos()</code>	Coordonnées <code>"x,y"</code>
<code>getCurrentCell()</code>	Cellule où tu te trouves (-1 si inconnue)
<pre>getPods() / getPodsMax() / getPodsPercent()</pre>	Pods actuels / max / % de remplissage
<pre>getCharacterLevel() / getCharacterName()</pre>	Niveau / nom du personnage
<pre>getLifePoints() / getMaxLifePoints()</pre>	Points de vie actuels / max / % (-1 si inconnus)

<code>getLifePointsPercent()</code>	
<code>isInFight()</code>	<code>true</code> si en combat
<code>isDead()</code>	<code>true</code> si le personnage est mort (en attente de résurrection)
<code>getStatus()</code>	Ce que fait le bot : <code>"ingame"</code> , <code>"moving"</code> , <code>"gathering"</code> , <code>"fight"</code> , <code>"dead"</code> , <code>"connecting"</code> , <code>"disconnected"</code> (session coupée, §4) ou <code>"offline"</code>
<code>getKamas()</code>	Kamas en poche (-1 si inconnu)
<code>getInventoryItemCount(gid)</code>	Quantité d'un objet dans le sac
<code>getItemName(gid)</code>	Nom lisible d'un objet (pratique pour les logs)

Se déplacer

Fonction	Effet
<code>goToCellId(cell)</code>	Marche jusqu'à une cellule de la carte courante
<code>changeMap("right")</code>	Change de carte dans une direction (<code>left</code> / <code>right</code> / <code>top</code> / <code>bottom</code> , les synonymes FR <code>gauche</code> / <code>droite</code> / <code>haut</code> / <code>bas</code> marchent aussi). Accepte aussi <code>"top(12)"</code> , <code>"left right"</code> , <code>"zaap(id)"</code> , <code>"zaapi(id)"</code> et <code>"havenbag"</code> (§7.5)
<code>travelTo(mapId)</code> / <code>travelTo(x, y)</code>	Voyage multi-cartes jusqu'à une carte : à pied, par le zaap de la carte ou via le havre-sac, selon le plus court
<code>saveZaap()</code>	Enregistre le zaap de la carte comme point de retour (respawn)

Récolter & interagir

Fonction	Effet
<code>gatherAll()</code>	Récolte toutes les ressources de la carte (filtrées par <code>ELEMENTS_TO_GATHER</code>)
<code>gatherOne()</code>	Récolte une seule ressource (la plus proche) ; <code>false</code> quand il n'y a plus rien. Idéal pour espacer les récoltes avec <code>delay()</code>
<code>goUseInteractive(cell)</code>	Utilise l'élément interactif d'une cellule précise

Combattre & survivre

Fonction	Effet
<code>fight()</code>	Attaque le groupe éligible le plus proche (selon tes filtres) et attend la fin du combat
<code>fightCell(cell)</code>	Privilégie un groupe proche d'une cellule donnée
<code>regenerate(pct)</code>	Remonte les PV jusqu'à <code>pct</code> % (consommables <code>REGEN_FOOD</code> , puis assis en attendant)

Inventaire & havre-sac

Fonction	Effet
<code>useInventoryItem(gid, count)</code> / <code>useItem(gid, count)</code>	Utilise <code>count</code> objets (ex. ouvrir des sacs de ressources) ; <code>count</code> omis = tous
<code>openBags()</code>	Ouvre d'un coup tous les sacs de ressources du sac
<code>deleteItem(gid, count)</code>	Détruit <code>count</code> objets (<code>count</code> omis = tous) - à ne pas confondre avec jeter
<code>dropItem(gid, count)</code>	Jette <code>count</code> objets

`enterHavenBag()`

Entre dans le havre-sac (ou en ressort si tu y es déjà)

Les modules (syntaxe `module:fonction()`) regroupent les fonctions par thème :

Module	Pour quoi faire	Exemples
<code>map:</code>	Lire la carte (monstres, PNJ, joueurs, interactifs, cellules), coffres et maisons à code	<code>map:getMonsters()</code> , <code>map:playersOnMap()</code>
<code>combat:</code>	Lire le combat et agir (pour <code>onTurn</code> , §7.12)	<code>combat:enemies()</code> , <code>combat:castOnCell(sort, cellule)</code>
<code>inventory:</code>	Inventaire et équipement	<code>inventory:items()</code> , <code>inventory:equip(gid)</code>
<code>bank:</code> / <code>exchange:</code>	Banque pilotée : ouvrir, retirer, déposer, kamas	<code>bank:open()</code> , <code>bank:withdraw(gid, qty)</code>
<code>trade:</code>	Échange avec un joueur présent sur ta carte, et mode « mule » pour ton équipe	<code>trade:withPlayer("Nom")</code> , <code>trade:putItem(gid, n)</code> , <code>trade:ready()</code>
<code>team:</code>	Ton équipe MizanBot et le groupe Dofus	<code>team:members()</code> , <code>team:inviteAll()</code>

<code>zaap: /</code> <code>zaapi:</code>	Zaaps et zaapis (Bonta / Brâkmar)	<code>zaap:go(mapId)</code> , <code>zaapi:go(mapId)</code>
<code>sale:</code>	Hôtel de Vente (vente, achat, mise à jour des prix)	<code>sale:ItemsOnSale()</code> , <code>sale:repriceAll()</code>
<code>memory: /</code> <code>sharedMemory:</code>	Une mémoire qui survit aux redémarrages (propre au perso / partagée entre tes bots)	<code>memory:set("banques", n)</code> , <code>memory:get("banques", "0")</code>
<code>quest:</code>	Quêtes et objectifs	<code>quest:isFinished(id)</code> , <code>quest:currentStep(id)</code>
<code>character:</code>	Identité, PV, énergie, points de caractéristiques	<code>character:breedName()</code> , <code>character:upgradeStat("vitality", 5)</code>
<code>stats:</code>	Compteurs de session et bilans (jour / semaine / mois)	<code>stats:summary("week")</code>
<code>data:</code>	Noms lisibles (monstres, objets, sorts, zones...)	<code>data:monsterName(id)</code> , <code>data:spellName(id)</code>
<code>job:</code>	Métiers	<code>job:level(jobId)</code>
	Requêtes web	

<code>http:</code>	<code>https</code> (droit Réponse automatique)	<code>http:get(url)</code>
<code>npc:</code>	PNJ (par id de modèle ou id sur la carte) : parler, lire la question, répondre, ouvrir la banque	<code>npc:npc(522), npc:reply(-1),</code> <code>npc:questionText()</code>
<code>chat:</code>	Écrire sur un canal du jeu (droit Réponse automatique)	<code>chat:sendGuildMessage("Salut")</code>
<code>craft:</code>	Craft pas à pas sur un établi déjà ouvert	<code>craft:PutItem(gid, 10),</code> <code>craft:Ready()</code>
<code>treasure:</code>	Chasse au trésor : démarrer, poser les drapeaux, creuser, abandonner (droit Récolte auto)	<code>treasure:isHunting(),</code> <code>treasure:huntDig()</code>
<code>global: /</code> <code>config: /</code> <code>developer:</code>	Les tables des scripts SnowBot (mémoire, réglages, messages du	<code>global:random(1, 5),</code> <code>config:setGatherList({ 38 })</code>

Fonctions avancées disponibles. En plus des verbes ci-dessus, MizanBot expose : le dialogue PNJ (`talkNpc` , `replyNpc` , `replyNpcAndChangeMap` , `leaveDialog`) ; les réponses se comptent **à partir de 1** ; `talkNpc` accepte l'id du PNJ sur la carte **ou** son id de **modèle**, §7.10), la banque au coffre (`putAllItems` , `putExistingItems` , `getAllItems` , `getExistingItems` , `depositItem`), le niveau de métier (`getJobLevel`), le **craft** à l'établi (`craftItem` , ou pas à pas avec `craft:`), la **chasse au trésor** (`treasure:`), l'**Hôtel de Vente** (`goToHdv` / `goToHdvBuy` + la table `sale:` , dont `sale:repriceAll()` qui repositionne juste sous le prix du marché celles de tes ventes qui sont plus chères), un **webhook Discord** (`sendWebhook`), les messages privés (`sendPrivateMessage`), tes propres bibliothèques (`require("nom")` , fichiers dans `%APPDATA%\MizanBot\scripts\lib`) et le passage à un autre script (`loadScript("nom")`). Les noms courants (`goToMapId` ...) sont reconnus et la plupart des scripts **SnowBot** sont prévus pour tourner sans modification (§7.13). Tu peux aussi écrire `map.getMonsters()` comme `map:getMonsters()`.

Indisponibles : l'achat/vente auprès d'un **PNJ marchand** (`sale:buyToPnj` , `sale:sellToPnj` , `sale:sellAllItemToPnj`), les **montures** (`mount:`) et le **Kolizéum**. Ces fonctions existent pour que ton script ne plante pas, mais elles ne font rien et le signalent une fois dans la console. Pour la chasse au trésor, les actions sont là, mais pas la résolution automatique des indices.

7.8 Un exemple en mode impératif

Ici on pilote « à la main », avec de la logique : récolter tout, prévenir quand le sac se remplit, et s'arrêter au niveau 100.

```
ELEMENTS_TO_GATHER = { 38 }    -- Blé

function move()
    if getCharacterLevel() >= 100 then
        printSuccess("Niveau 100 atteint, on arrête !")
```

```

    return false          -- renvoyer false = fin du script
end

gatherAll()              -- récolte tout le blé de la carte

if getPodsPercent() >= 90 then
    printWarning("Sac presque plein (" .. getPodsPercent() .. "%)")
end

changeMap("right")       -- passe à la carte de droite, puis on
end

```

Variante plus « humaine » : récolter **une ressource à la fois**, avec une pause entre chaque :

```

function move()
    while gatherOne() do    -- une ressource...
        delay(2000)        -- ...puis ~2 secondes de pause
    end
    changeMap("right")
end
end

```

Tu disposes de tout le **Lua standard** (`if`, `for`, `while`, variables, `string`, `table`, `math`, `..` pour concaténer...), de l'heure (`os.time()`, `os.clock()`, `os.date()`) et de `json`. Ce qui n'est **pas** disponible : l'accès aux fichiers et au système (`io`, `os.execute` ...). Le moteur tourne en bac à sable pour ta sécurité : `require` et `dofile` ne chargent que les fichiers de ton dossier de scripts, par leur **nom**, jamais un chemin du disque.

7.9 Bon à savoir (comment le moteur se comporte)

- `move()` est appelée en boucle. En déclaratif, renvoie ton itinéraire ; en impératif, fais tes actions (et `return false` pour arrêter).
- **Le combat est automatique... si tu l'as activé.** Si un combat démarre (y compris une **agression** pendant la récolte), le script se met en pause, le

module Combat (§6) joue le combat, puis la récolte **reprend** toute seule. Pour ça, coche **Jouer automatiquement mes tours de combat** (onglet **Combat**, décoché par défaut) et règle tes sorts : sinon personne ne joue tes tours et le script attend la fin du combat. **Exception** : si ton script définit `onTurn()`, c'est lui qui joue tes tours (§7.12).

- **La mort n'arrête pas le script.** Avec **Résurrection auto (Phénix)** (onglet **Combat**, activée par défaut), le bot ressuscite au Phénix puis reprend là où il en était.
- **Le voyage est automatique et malin.** Pour rejoindre une carte, le bot choisit le plus court : à pied, par le **zaap de la carte** où il se trouve, ou via le **havre-sac** puis un zaap (parmi ceux que ton personnage connaît). Perdu hors de ton itinéraire sans chemin à pied ? Il tente le zaap ou le havre-sac tout seul pour revenir. Pas besoin de `NEAREST_ZAAP` (variable des scripts SnowBot, ignorée). Le bot tient compte de la carte où mène réellement chaque sortie (trajets plus courts) ; s'il arrive ailleurs que prévu, il recalcule son trajet depuis là.
- **Zones sans havre-sac** : à **Incarnam**, le bot ne tente jamais le havre-sac ; dans certaines instances, il s'en rend compte et n'insiste pas. Il marche ou prend un zaap à la place.
- **Carte introuvable ?** Si la carte visée n'existe pas dans la carte du monde du bot (le plus souvent un **id de carte faux**, ou une carte de même coordonnée dans une autre zone), il ne tente ni zaap ni havre-sac et te le dit dans la console : mets plutôt la coordonnée `"x,y"` dans ton étape. Coincé sur une carte qu'il ne sait pas parcourir (instance, certains intérieurs) : il patiente un peu, puis arrête le script.
- **Trajet vraiment impossible ?** Après 3 échecs vers la même carte (ou 5 tentatives pour rejoindre ton itinéraire), le script s'arrête avec une notification et le planning se met en pause jusqu'à la fin de la plage (§6). La console Lua (`[trajet]`) indique la carte en cause.
- **Les actions sont bloquantes et propres.** Un `goToCellId` / `gatherAll` / `fight` / `travelTo` attend le serveur ; le bouton **Arrêter** interrompt proprement (pas de coupure brutale). Un **HDV** resté ouvert est refermé automatiquement avant de repartir.
- **Les droits sont vérifiés au moment d'agir.** La récolte (et l'inventaire, la

banque, les échanges, le craft, la chasse au trésor, la validation de quêtes) exige la permission **Récolte auto**, le combat **Combat auto**, l'équipe (et le lancement d'autres comptes depuis un script) **Multi**, l'HDV **Hôtel de Vente**, les messages, webhooks et requêtes web **Réponse automatique** (§10). Sans le droit, l'action est refusée avec un avertissement, et le reste du script continue. Avec l'offre gratuite, un personnage au-delà du **niveau 30** ne peut pas lancer de script.

- **Mets tes actions dans** `move()` (ou dans un hook, §7.12), pas en tête de fichier : au chargement du script, ton personnage n'est pas forcément encore identifié.
- **Session de jeu perdue ?** Si le serveur te déconnecte (inactivité...), le script s'arrête proprement (`stopped("disconnected")`, §7.12) et reste **mémorisé** : **Reconnecter** (§4) reconnecte le perso et **relance** ce script au retour en jeu. En mode Socket, une simple coupure réseau est reprise **toute seule** : le script est suspendu puis relancé après la reconnexion. Dans tous les cas c'est un **nouveau départ** du script (compteurs de session remis à zéro) : garde dans `memory:` ce qui doit être conservé.
- **Sécurité licence.** Si ta session de licence est coupée côté serveur (expiration, révocation...), tous les scripts s'arrêtent automatiquement.

7.10 Trouver les ID (ressources, monstres, objets)

Les scripts utilisent trois familles d'identifiants Dofus :

Tu veux...	Utilise...	Dans...
Choisir quoi récolter	ElementTypeId (type de ressource)	<code>ELEMENTS_TO_GATHER = { ... }</code>
Filtrer les combats	nom ou genericId de monstre	<code>FORBIDDEN_MONSTERS</code> / <code>FORCE_MONSTERS</code>
Manipuler	gid (id d'objet)	<code>getInventoryItemCount</code> / <code>useItem</code> / <code>deleteItem</code> / <code>dropItem</code> /

Exemples courants : Blé `38`, Fer `17`, Frêne `1` (récolte) ; Bouftou `36` (monstre) ; Sac de blé `7941` (objet).

 **Toutes les listes complètes** (ressources par métier, métiers, interactifs, monstres, objets, avec les fichiers à télécharger) sont dans l'**Annexe** en bas de ce guide.

 En jeu, `/cmd print(getItemName(7941))` affiche dans la console le nom qui correspond à un gid, et l'onglet **Inventaire** (§6) affiche le **gid** de chaque objet de ton sac.

 Les autres ids se trouvent depuis la **Console** du bot (§6) : `/mapid` (id de la carte), `/cellid` (ta cellule), `/id NOM` (un joueur, ou un PNJ avec son **id de modèle**). Pour lister les PNJ de la carte : `/cmd for _, n in ipairs(map:getNpcs()) do print(n.name .. " : " .. n.id .. " (modèle " .. n.npcId .. ")") end`. `talkNpc` accepte l'un ou l'autre ; dans un script, préfère l'**id de modèle** (`npcId`) : il désigne toujours le même PNJ, alors que l'id sur la carte (`id`, négatif) ne vaut que pour cette carte (et que le nom dépend de la langue). Le survol d'un PNJ dans l'onglet **Carte** affiche aussi son modèle, et **Afficher les numéros de cellule** y montre les cellules.

7.11 Petits pièges fréquents

Symptôme dans la console	Cause probable
<code>attempt to call a nil value</code>	Nom de fonction mal orthographié, ou fonction inconnue de MizanBot. Vérifie le nom dans la référence de l'API.
« le script doit définir une fonction move() »	Il manque <code>function move() ... end</code> (facultative seulement si ton script ne fait que jouer les combats avec <code>onTurn</code> ou réagir aux

	messages, §7.12).
« aucune étape pour la carte courante »	Ta position ne correspond à aucune étape et le bot n'a trouvé aucune carte de ton itinéraire à rejoindre : il reste sur place. Vérifie tes <code>map = "x,y"</code> (coordonnées bien écrites, bonne zone). Quand il sait rejoindre ton trajet, la console affiche plutôt « hors trajet (...) → rejoint la carte de route la plus proche » et tu n'as rien à faire.
« ... ressource(s) récoltable(s) [ElementTypeId présents ici : ...] mais aucune ∈ ELEMENTS_TO_GATHER ... »	Ton filtre ne contient pas le bon id : le message liste les ids présents sur la carte, ajoute celui qui t'intéresse.
« cible HORS du world-graph : l'id de l'étape est sans doute faux »	L'id de carte de ton étape ne correspond pas à la carte voulue (souvent une carte de même coordonnée dans une autre zone) : mets la coordonnée <code>"x,y"</code> à la place (§7.9).
« PNJ introuvable sur la carte »	L'id de modèle passé à <code>talkNpc</code> / <code>npc:npc</code> ne correspond à aucun PNJ de la carte. Vérifie-le avec <code>/id NOM</code> (§7.10).
« ... : fonction SnowBot sans équivalent dans MizanBot — appel ignoré » (ou « pas encore disponible dans MizanBot »)	Normal avec un script SnowBot : cette fonction ne fait rien et le reste du script continue (§7.13).
« ... : refusé par sécurité — appel ignoré »	Le script voulait lire un mot de passe, un jeton, un code du Bouclier, ou gérer des comptes : c'est volontairement bloqué. Retire cet appel.

« Session de jeu perdue (...) : script ARRÊTÉ, conservé pour la reconnexion »	Le serveur a coupé la session : clique Reconnecter (§4).
Boucle HDV : 1er lot lu deux fois, dernier jamais lu	<code>sale:GetItemGID</code> / <code>GetItemGUID</code> / <code>GetItemPrice</code> / <code>GetItemQuantity</code> comptent à partir de 1 (comme SnowBot ; 0 est encore accepté comme 1er lot). Écris <code>for i = 1,</code> <code>sale:ItemsOnSale() do</code> .
« replyNpc(0) hors plage — index 1-based ... »	Les réponses PNJ se comptent à partir de 1 : <code>replyNpc(1)</code> = la première réponse.
Erreur de syntaxe près de <code>goto</code>	<code>goto</code> est un mot réservé du Lua : écris <code>zaap:go(mapId)</code> / <code>zaapi:go(mapId)</code> .
« Récolte refusée : permission « Récolte auto » requise » / « Combat refusé : permission « Combat auto » requise »	Le droit Récolte auto / Combat auto n'est pas inclus dans ta licence (§10).

7.12 Aller plus loin : hooks et combat scripté

Un **hook** est une fonction que **tu** définis dans ton script et que MizanBot **appelle tout seul** au bon moment. Tous sont facultatifs.

Hook (dans ton script)	Quand MizanBot l'appelle
<code>onTurn()</code> (ou <code>fightManagement()</code> des scripts SnowBot)	À chacun de tes tours de combat. Le tour se termine tout seul quand la fonction se termine. Temps max : <code>TURN_TIME_LIMIT</code> (20 s par défaut)

<code>onPlacement()</code>	Au début du placement. Renvoie <code>false</code> si tu gères toi-même le « prêt » (<code>combat:ready()</code>) ; sinon le bot se met prêt tout seul
<code>onFightEnd(result)</code>	Après chaque combat (<code>result.won</code> , <code>result.loot</code> ...)
<code>stopped(reason)</code>	Une fois, quand le script s'arrête. <code>reason</code> vaut par exemple <code>"ended"</code> (<code>move()</code> a renvoyé <code>false</code>), <code>"user"</code> (bouton Arrêter), <code>"script"</code> (<code>stopScript()</code>), <code>"error"</code> , <code>"stuck"</code> (trajet bloqué), <code>"planning"</code> , <code>"disconnected"</code> (session perdue, script repris par Reconnecter) ou <code>"socket"</code> (coupure en mode Socket, script repris après la reconnexion). Dedans, contente-toi de <code>print</code> , <code>sendWebhook</code> , <code>memory:</code> et de lectures (2 s max)
<code>onPrivateMessage(from, text, id)</code>	À chaque message privé reçu en jeu (<code>id</code> = id de l'expéditeur)
<code>messagesRegistering()</code>	Une fois au démarrage : c'est là que tu declares tes <code>registerMessage(...)</code> (voir plus bas)
<code>phenix()</code>	Tant que ton perso est fantôme : renvoie un trajet vers la statue de ton choix (étape { <code>map = "x,y"</code> , <code>phenix = cellule</code> }). Sans étape pour la carte, le bot fait la résurrection lui-même (§6)
<code>prefightManagement(...)</code> / <code>challengeManagement(...)</code>	Callbacks de placement et de défi des scripts SnowBot, appelés seulement si <code>onPlacement()</code> est absent. Le bot fait ensuite son pré-combat habituel, sauf si tu

Exemple : ta propre IA de combat

```

MON_SORT = 0  -- remplace par l'id de ton sort d'attaque

function onTurn()
  for _, e in ipairs(combat:enemies()) do
    if e.alive and combat:canCast(MON_SORT, e.cell) then
      combat:castOnCell(MON_SORT, e.cell)  -- lance le sort sur le 1e
      return                               -- fin de tour automatique
    end
  end
  combat:playTurn()  -- personne à portée : le bot joue ce tour avec
end

```

- Tant que `onTurn()` existe, il **remplace** le combat automatique de l'onglet Combat.
- Si ton `onTurn()` plante avant d'avoir agi et que **Jouer automatiquement mes tours de combat** est coché, le bot joue ce tour à ta place. Pour un `fightManagement()` hérité de SnowBot qui plante, le bot joue ce tour **et tous les suivants** avec l'onglet Combat.
- `move()` devient **facultative** : un script qui ne contient que `onTurn()` se contente de jouer tes combats.
- 💡 Pour connaître l'id de tes sorts, tape dans la Console : `/cmd for _, s in ipairs(combat:getSpells()) do print(s.id .. " = " .. data:spellName(s.id)) end`

Réagir à un message privé

```

function onPrivateMessage(from, text)
  printWarning("MP de " .. from .. " : " .. text)
  notify("MP de " .. from)  -- notification dans MizanBot
end

```

Pour les avancés : `registerMessage("NomDuMessage", fonction)` appelle ta fonction à chaque message du jeu de ce type, et `waitFor("NomDuMessage")` attend qu'il arrive (avec un délai max). Tu peux donner le nom Dofus 3 du message ou son nom Dofus 2 (celui des scripts SnowBot). Les règles de nommage et la forme des données reçues sont expliquées dans la **référence de l'API** (espace client → Documentation) ; elle ne contient pas la liste de tous les messages du jeu.

7.13 Tu viens de SnowBot ?

La plupart des scripts écrits pour **SnowBot** se lancent **sans modification** :

Charger le `.lua`, puis **Lancer**. Leurs tables (`global:`, `config:`, `map:`, `npc:`, `sale:`, `craft:`, `treasure:`, `developer:`, `fightAction:`, `fightCharacter:` ...), leurs fonctions spéciales (`move`, `bank`, `phenix`, `fightManagement`, `prefightManagement`, `challengeManagement` ...) et leurs globals (`PLANNING`, `FIGHT_KICK`, `GATHER` ...) sont reconnus.

Les différences à connaître

Sujet	Chez SnowBot	Dans MizanBot
Seuil de banque	Réglé dans l'interface (90 %)	<code>MAX_PODS</code> (95 % si absent) : ajoute <code>MAX_PODS = 90</code>
Taille max d'un groupe	8 si absent	Pas de limite si absent : ajoute <code>MAX_MONSTERS = 8</code>
Expulsion au placement, refus des invitations de guildes	Par défaut	Seulement avec <code>FIGHT_KICK</code> / <code>IGNORE_REQUEST_GUILD</code> (§7.4)
<code>move()</code>	Appelée une fois par carte	Appelée en boucle : un <code>print</code> ou un compteur en tête de <code>move()</code> se répète
	Arrêt du	Le bot rejoint la carte la plus

Carte sans étape	script	proche de ton trajet
Listes lues dans les messages du jeu	<code>for i = 0, t.Count - 1</code>	Commencent à 1 : <code>for i = 1, t.Count</code>
PNJ	Id de modèle	Modèle ou id sur la carte (§7.10) ; <code>replyNpc(1)</code> = 1re réponse
<code>global:addInMemory</code>	Mémoire du bot	Gardée jusqu'à la fermeture de MizanBot ; pour plus longtemps : <code>memory:</code>
Fichiers (<code>dofile</code> , <code>loadAndStart</code>)	Chemin sur le disque	Seul le nom du fichier compte (dossier de scripts de MizanBot)

Ce qui ne marche pas (la fonction est ignorée et la console l'indique une fois) :

- **refusé par sécurité** : lire un mot de passe, un jeton ou un code du Bouclier, créer un compte ou un personnage, gérer les proxys, envoyer des messages libres au serveur ;
- **non disponible dans MizanBot** : la boutique des **PNJ marchands**, les **montures** ;
- **sans équivalent** : Kolizéum, questions à l'écran (`askQuestion` ...), anti-moderateur SnowBot (utilise `MODERATOR_STOP` / `MODERATOR_WEBHOOK` , §7.4), résolution automatique des indices de chasse au trésor.

Le réglage `AI_FILE` (fichier d'IA SnowBot) est ignoré **sans message** : règle ton combat dans l'onglet **Combat** ou avec `onTurn` (§7.12).

 Toutes les différences avec SnowBot (HDV, PNJ, mémoire, combat, paquets...) sont détaillées dans la **référence de l'API** (espace client → **Documentation**, catégorie « Compatibilité SnowBot »).

8. Réglages

Dans **Réglages** (tes préférences sont mémorisées) :

Rubrique	Ce que tu y règles
Langue	FR / EN / ES
Apparence	Le thème (couleur d'accent : Bleu, Violet, Émeraude, Cramoisi, Ambre, Sombre (opaque))
Interface	Interface façon SnowBot : une autre disposition, réversible ; MizanBot propose Redémarrer maintenant pour l'appliquer (sinon, au prochain lancement)
Audio	Sons d'interface , Volume des sons (curseur, actif quand les sons sont activés), Musique d'ambiance
Effets	Particules d'arrière-plan
Mods natifs en jeu	Injecter les mods natifs (no-anim, mode tactique... et plus de fenêtre « Inactivité » de Dofus quand ton perso reste immobile ; prend effet au prochain lancement du client) et Relance automatique : relance un client qui se ferme de façon inattendue (3 fois au plus en 3 minutes, §4) et reconnecte tout seul un compte Socket après une coupure réseau
Confidentialité	Mode Recording / Stream : floute tes informations personnelles dans l'interface pour filmer ou streamer. Les deux options Capture... juste en dessous servent uniquement au diagnostic demandé par le support : laisse-les sur Off
Archimonstres	Recevoir les MP Discord d'archimonstres quand un de tes personnages en repère un. Désactivé, tu ne dois plus recevoir ce MP (prise en compte côté serveur en cours de déploiement : en attendant, il peut encore arriver), mais la détection est quand même partagée, de façon anonyme , dans le salon public du Discord ; l'alerte dans MizanBot

	reste active dans tous les cas
Discord	Statut Discord « Joue à MizanBot » : affiche MizanBot dans ton statut Discord comme un jeu (activé par défaut).  Tes contacts Discord le voient ; Discord doit être lancé. Et Rejoindre le Discord (support, annonces des mises à jour, entraide et partage de scripts)
Vue / densité	Taille d'affichage : Compact / Normal / Grand

9. Mises à jour automatiques

MizanBot se met à jour **tout seul**. Quand une nouvelle version est publiée, au lancement l'application **télécharge et installe** la mise à jour, puis redémarre : tu n'as **rien à faire**. Les nouveautés de la version s'affichent pendant l'installation.

Une mise à jour est **obligatoire** : tant qu'elle n'est pas installée, l'accès à l'application est bloqué (c'est voulu, pour que tout le monde soit sur la même version).

 Après une **mise à jour de Dofus**, la fenêtre **Préparation des cartes** (§2) peut réapparaître une fois : clique **Continuer en arrière-plan** pour continuer à utiliser MizanBot pendant ce temps.

10. Tes droits (permissions)

Ta licence inclut un ensemble de **fonctionnalités**. Si une fonctionnalité n'est pas incluse, son bouton est **grisé** et un **bandeau** (ou l'info-bulle du bouton) explique pourquoi. Les principaux droits :

Droit	Débloque
	La récolte automatique de ressources (et, dans les scripts,

Récolte auto	l'inventaire, la banque, les échanges, le craft, la chasse au trésor et la validation de quêtes)
Combat auto	Le combat automatique (rotation de sorts, pré-combat, auto level-up, IA scriptée <code>onTurn</code> ...)
Mode tactique / Mode no-anim	Les mods natifs de confort en combat, en mode MITM (<i>Collapse</i> et <i>Déplacements instantanés</i> dépendent de Mode no-anim)
Multi	Lancer plusieurs comptes en même temps (et les équipes ; dans les scripts, lancer ou reconnecter un autre compte)
Mizan Adapter	Créer des proxys depuis tes cartes réseau
Proxy réseau	Utiliser des proxys externes SOCKS5 / HTTP
Hôtel de Vente	L'onglet HDV et les fonctions HDV des scripts
Réponse automatique	La messagerie : envoyer un message en jeu depuis la barre de la Console (§6), la réponse automatique aux MP et le relais Discord ; dans les scripts, les messages (MP, <code>chat:</code>), les webhooks et <code>http:</code>
Mode Socket	Lancer un compte sans ouvrir Dofus (bouton Socket , §4), y compris via le planning, Lancer tout , l'équipe et la relance automatique. Le mode MITM (bouton Lancer) reste inclus dans toutes les licences

Retrouve le détail dans l'onglet **Mon compte** : chaque permission y est marquée **Actif** ou **Inactif**, avec son échéance (« Suit l'expiration de la licence », une date, ou « Non inclus dans la licence ») ; une permission peut expirer plus tôt que la licence. Si une permission est ajoutée ou retirée pendant que MizanBot tourne, une notification te prévient ; un compte lancé grâce à une permission retirée

(proxy, adaptateur, multi, Socket) est arrêté.

Offre gratuite (récolte seule) : le botting est réservé aux personnages jusqu'au **niveau 30**. Au-delà, un bandeau l'indique dans l'onglet **Lua** du personnage (onglet **Bots**), où **Lancer** est grisé (un script en cours s'arrête), et le planning ne relance pas ce personnage.

11. Ton espace client (le portail web)

En plus de l'application, tu as un **espace client** dans ton **navigateur** (le lien est dans ton Discord / ton mail de livraison). C'est là que tu **télécharges MizanBot**, **gères ta licence**, et **pilotes tes sessions et tes bots à distance**.

Se connecter

- La connexion se fait avec ta **clé de licence uniquement** : **pas d'e-mail ni de mot de passe**. Colle ta clé → **Se connecter**.
- Si la clé est **invalidé, révoquée, en pause ou bannie**, un message clair te l'indique (avec la raison pour une révocation / un bannissement). Après **trop de tentatives**, patiente une minute.
- **Clé perdue ?** Contacte un administrateur : il n'y a pas de récupération automatique.
- Un sélecteur **FR / EN** (en haut à droite) est disponible sur **toutes les pages**.

Le menu « Ma licence »

Onglet	Ce que tu y fais
Vue d'ensemble	Ta clé, ton statut, ton échéance, tes fonctionnalités incluses, lier ton Discord
Session active	Voir en direct où ta licence est connectée + couper la connexion
Mes bots	Voir tes bots en jeu en temps réel + en déconnecter un à distance

Verrou HWID	Voir et réinitialiser le verrou d'appareil (changer de PC)
Téléchargements	Récupérer MizanBot et les fichiers inclus dans ta licence
Documentation	Lire les guides publiés par l'équipe (<i>onglet présent seulement s'il y a du contenu</i>)

Et sur chaque page : un bouton **Se déconnecter**, le **sélecteur de langue**, un **bandeau** si ton Discord n'est pas encore lié (avec « **Lier maintenant** »), les **annonces de l'équipe** (que tu peux masquer), et un lien « **Rejoindre notre Discord** ».

Vue d'ensemble

- **Ta clé de licence** : **Afficher / Masquer** la clé complète, et la **Copier** en un clic.
- **Ton abonnement** : jours restants avant expiration, **date d'activation**, **date d'expiration** (ou « À l'activation » / « Jamais »), et l'état du **verrou d'appareil**.
- **Discord** : **Lier** ou **Délier** ton compte Discord. Lié, tu reçois les **DM d'abonnement** (renouvellement, expiration à venir, changements). Délié, tu ne les reçois plus.
- **Ce que ta licence débloque** : la liste de **tes fonctionnalités incluses** (voir §10).
- **Offres & tarifs** : un **comparatif** de toutes les formules (inclus / non inclus par fonctionnalité) ; ton offre actuelle est **surlignée**. C'est **informatif** : pour **renouveler ou upgrader**, passe par l'équipe sur **Discord**.
- Un **bandeau** t'alerte si ta licence est **en pause** ou **révoquée** (avec la raison si renseignée).

Session active

- Voir **en direct** (la page se rafraîchit toute seule) **où ta licence est utilisée** : statut, **pays / ville**, **adresse IP**, **application** cliente + version, heure de **démarrage**, et **dernier signe de vie** (heartbeat).
- **Terminer ma session** : **coupe la connexion à distance**. L'application s'arrête

jusqu'à ce que tu la relances (ce qui ré-authentifie ta licence depuis ton appareil). Idéal si tu vois un **emplacement que tu ne reconnais pas**.

- « **Aucune session active** » = rien n'est connecté ; lance l'application pour démarrer une session.

Mes bots

- La liste **en temps réel** de tes bots actuellement en jeu : chaque carte montre le **personnage**, le **serveur**, le **niveau**, la **position** (carte + coordonnées), le remplissage en **pods** et les **kamas**.
- **Déconnecter** un bot à **distance** (comme un **Stop** dans l'application) : tu pourras le **relancer** ensuite. Une confirmation est demandée.
- « **Aucun bot en ligne** » = aucun bot ne tourne ; lance-en un depuis MizanBot, il apparaîtra ici tout seul.

Verrou HWID

- Voir si ta licence est **verrouillée** sur ton PC, et si l'**auto-réinitialisation** est **disponible** (ou la date à laquelle elle le redeviendra).
- **Réinitialiser mon HWID** : détache ta licence de l'appareil actuel pour **changer d'ordinateur** ; elle se reliera au **prochain** PC lancé. Limité à **une fois tous les 7 jours** ; au-delà, demande une réinitialisation exceptionnelle à un **administrateur**.

Téléchargements

- La liste des **fichiers inclus dans ta licence** : **version**, **taille**, **description**, et un bouton **Télécharger**.
- Certains fichiers exigent une **permission précise** (indiquée par un badge) : ils n'apparaissent que si ta licence la possède.
- Le lien de téléchargement est **privé** (lié à ta session) : inutile de le partager, il ne sert qu'à toi.

Documentation

- Les **guides publiés par l'équipe**, en **lecture seule**, avec la **date de dernière mise à jour**. L'onglet n'apparaît que **si du contenu a été publié**.

12. Dépannage

Souci	Que faire
« Mise à jour requise » au lancement	Laisse la mise à jour se faire (elle est automatique) ; en cas d'échec réseau, clique Réessayer .
Une fonctionnalité est grisée	Ta licence ne l'inclut pas (voir le bandeau ou l'info-bulle + Mon compte). Contacte le support pour l'ajouter.
Un message tapé dans la barre de la Console n'est pas envoyé (il reste dans la case)	Envoyer des messages en jeu demande la permission Réponse automatique (§10, voir Mon compte). Les commandes qui commencent par  ne sont pas concernées.
Carte du compte : « Déconnecté par le serveur (inactivité) » (ou « Déconnecté (...) »)	Le serveur a coupé la session, ou tu as quitté le jeu sans y revenir (en MITM, Dofus reste ouvert). Clique Reconnecter (sur la carte du compte ou en haut de l'onglet Bots) : retour en jeu sur le même perso, et le script reprend. Rien n'est reconnecté automatiquement après une déconnexion par le serveur (§4).
Console Lua : « De retour en jeu : le script arrêté à la perte de session n'est PAS repris automatiquement »	Tu es revenu en jeu depuis la fenêtre Dofus plutôt qu'avec Reconnecter : relance ton script avec Lancer (onglet Lua).

« Reconnecté sur un autre personnage (...) : script non repris »	Le compte est revenu en jeu sur un autre perso : reconnecte-toi sur le bon, puis Lancer .
« Reconnexion déjà en cours pour ce compte. »	Attends le retour en jeu : le script reprend tout seul. Arrêter annule une reconnexion en attente.
Le bouton Socket est grisé	Survole-le : soit ta licence n'inclut pas Mode Socket (§10), soit ta licence n'est pas encore connectée. Le bouton Lancer (mode MITM) reste disponible.
Mode Socket : « connexion socket perdue — reconnexion auto dans ... s »	Coupure réseau ou proxy : MizanBot se reconnecte tout seul et reprend le script. Reconnecter écourté l'attente. Si c'est « reconnexion suspendue » (ou « connexion socket perdue. » quand Réglages → Relance automatique est désactivée) : vérifie ton proxy (Tester), puis relance le compte avec le bouton Socket (Lancer le relancerait en MITM, avec la fenêtre Dofus).
Mode Socket : « session précédente encore tenue par le serveur — nouvel essai dans ... s »	Normal juste après une coupure : le serveur libère l'ancienne session et MizanBot réessaie seul.
Un mod (no-anim / tactique...) ne s'applique pas	Relance le client Dofus concerné (l'injection se fait au (re)lancement). En mode Socket (sans fenêtre de jeu), les mods n'ont pas d'effet : c'est normal.

« Proxy indisponible (adaptateur éteint ?) » au lancement

Vérifie que le proxy est joignable (onglet **Proxys** → **Tester**) ou que l'interrupteur de l'adaptateur est activé (onglet **Adaptateurs**).
Message « Permission ... requise » : ta licence n'inclut pas **Proxy réseau / Mizan Adapter** (§10).

Jeu bloqué au chargement avec un proxy, et dans la **Console** (ou `%APPDATA%\MizanBot\logs\transport-*.log`) : « le PROXY a REFUSÉ le tunnel vers le serveur de JEU »

Ton proxy ne laisse passer que le web (le **Tester** passe, le jeu non).
Utilise un proxy qui autorise **tous les ports**, ou demande à ton fournisseur de débloquer le port indiqué dans le message.

Au lancement : « Ce compte a déjà un Bouclier de sécurité Ankama actif dont MizanBot n'a plus le certificat »

Réinitialise ou désactive le Bouclier sur ton compte Ankama (Sécurité / appareils de confiance), puis relance : cette fois, MizanBot mémorise l'autorisation.

Fenêtre **Localise ton Dofus.exe** au lancement, ou « Planning : Dofus.exe introuvable »

MizanBot ne trouve pas Dofus : lance un compte à la main avec **Lancer** et sélectionne ton `Dofus.exe` (l'emplacement est mémorisé). Si tu fermes la fenêtre, le lancement est annulé. Un compte en mode **Socket** se lance sans, mais MizanBot doit quand même trouver Dofus pour lire les cartes (§2) : si Dofus est installé ailleurs qu'à l'emplacement habituel, fais un **Lancer** une fois pour le localiser, puis reviens au mode Socket avec le bouton

	Socket.
« Client fermé (crash) » / « trop de crashes coup sur coup — relance auto suspendue »	Dofus s'est fermé de façon inattendue. Avec Relance automatique (§8), il est relancé (3 fois au plus en 3 minutes) ; au-delà, vérifie que Dofus se lance normalement, puis relance le compte à la main.
Notification « bot figé (aucune action depuis un moment) »	L' Anti-blocage (onglet Paramètres , §6) n'a pas réussi à débloquer le bot. Regarde la console Lua et le jeu (dialogue ouvert ? personnage coincé ?), puis relance le script. Tu peux cocher Redémarrer le script si toujours bloqué , ou augmenter le Délai avant blocage si ton script fait de longues pauses.
Notification « script arrêté (bloqué) » (+ « planning en pause jusqu'à la fin de la plage »)	Le trajet est impossible (voir §7.9). La console Lua ([trajet]) indique la carte en cause ; corrige ton itinéraire. Si elle parle de « cible HORS du world-graph » (carte inconnue du réseau de trajets de MizanBot), l'id de carte de l'étape est sans doute faux : mets plutôt sa coordonnée "x,y" . Le planning reprend à la plage suivante, ou dès que tu cliques Lancer .
Fenêtre « Inactivité » de Dofus	Vérifie que Réglages → Injecter les mods natifs est activé, puis

	relance le client Dofus.
Ton antivirus bloque ou supprime MizanBot	C'est un faux positif : relance <code>MizanBot-Setup.exe</code> et réponds Oui à l'exclusion Windows Defender (§1), ou ajoute une exclusion pour <code>%LOCALAPPDATA%\Mizan</code> dans ton antivirus.
Un script Lua ne démarre pas / s'arrête aussitôt	Lis la console Lua (§7.1) : elle indique la cause exacte (fonction inconnue, <code>move()</code> manquante, droit absent, limite de niveau de l'offre gratuite...). Voir §7.11, et §7.13 pour un script SnowBot. Pendant une reconnexion , le lancement est refusé : attends le retour en jeu.
Bandeau « Offre gratuite : ce personnage dépasse le niveau 30 »	L'offre gratuite limite le botting aux personnages jusqu'au niveau 30 (§10). Utilise un perso plus bas ou passe à une offre payante.
Changement de PC (« Cette licence est liée à un autre appareil »)	Espace client → Verrou HWID → Réinitialiser mon HWID (1x / 7 jours) ; sinon contacte le support.
Une connexion que je ne reconnais pas	Espace client → Session active - > Terminer ma session pour couper l'accès immédiatement.

13. Support

Une question, un souci, ou besoin d'ajouter une fonctionnalité à ta licence ?
Rejoins le **Discord** : l'icône **Discord** en haut à droite de la fenêtre, **Réglages** → **Rejoindre le Discord**, le lien **Besoin d'aide ? Rejoindre le Discord** sur l'écran de connexion, ou le lien de ton espace client. L'équipe et la communauté y répondent.

Bon jeu ! 🎮

Annexe - Identifiants Dofus (pour les scripts Lua)

Tous les **identifiants** utiles pour écrire tes scripts Lua (§7). Chaque liste correspond à un paramètre de script :

Tu veux...	Utilise l'identifiant...	Dans...
Choisir quoi récolter	ElementTypeId (type de ressource)	<code>ELEMENTS_TO_GATHER = { ... }</code>
Filtrer les combats	nom ou genericId de monstre	<code>FORBIDDEN_MONSTERS</code> / <code>FORCE_MONSTERS</code>
Manipuler l'inventaire	gid (id d'objet)	<code>getInventoryItemCount</code> / <code>useInventoryItem</code> / <code>deleteItem</code> / <code>dropItem</code> / <code>AUTO_DELETE</code>

💡 En jeu, `getItemName(gid)` affiche le nom correspondant à un id dans la console Lua : pratique pour vérifier une valeur.

Volumes de référence : Interactifs 389, Monstres 2 835, Objets 19 029.

A.1 Ressources récoltables par métier (ElementTypeId)

Valeurs à mettre dans `ELEMENTS_TO_GATHER`. Exemple : pour ne récolter que le blé et l'orge → `ELEMENTS_TO_GATHER = { 38, 43 }`. Laisse la liste vide (`{ }`) ou omets-la pour tout récolter (seulement les vraies ressources : jamais les portes, zaaps ou ateliers).

 **Paysan** - Blé `38`, Orge `43`, Avoine `45`, Houblon `39`, Lin `42`, Seigle `44`, Quisnoa `289`, Riz `111`, Malt `47`, Chanvre `46`, Maïs `260`, Millet `261`, Frostiz `134`

 **Alchimiste** - Ortie `254`, Sauge `255`, Trèfle à 5 feuilles `67`, Menthe Sauvage `66`, Orchidée Freyesque `68`, Edelweiss `61`, Pandouille `112`, Ginseng `256`, Belladone `257`, Mandragore `258`, Perce-neige `131`, Salikrone `288`

 **Mineur** - Fer `17`, Cuivre `53`, Bronze `55`, Kobalte `37`, Manganèse `54`, Étain `52`, Argent `24`, Silicate `114`, Bauxite `26`, Or `25`, Dolomite `113`, Obsidienne `135`, Écume de mer `291`

 **Bûcheron** - Frêne `1`, Châtaignier `33`, Noyer `34`, Chêne `8`, Bombu `98`, Érable `31`, Oliviolet `101`, Pin `401`, If `28`, Bambou `108`, Merisier `35`, Noisetier `259`, Ébène `29`, Kaliptus `121`, Charme `32`, Bambou sombre `109`, Orme `30`, Bambou sacré `110`, Aquajou `294`, Tremble `133`

 **Pêcheur** - Goujon `75`, Greuvette `71`, Truite `74`, Crabe `77`, Poisson pané `78`, Poisson-Chaton `76`, Carpe d'lem `79`, Sardine Brillante `81`, Brochet `263`, Kralamoure `264`, Anguille `265`, Dorade Grise `266`, Perche `267`, Raie `268`, Lotte `269`, Bar Rikain `271`, Morue `272`, Tanche `273`, Espadon `274`, Patelle `297`, Poisskaille `132`, Pichon d'encre `365`, Requin Marteau-Faucille `270`

 **Autres** - Puits `84`

💡 Un id ne marche pas ? Si la carte a des ressources mais qu'aucune ne correspond à ton filtre, la console Lua **liste les ElementTypeld présents** sur la carte (§7.11).

A.2 Identifiants des métiers (JobID)

Référence pratique (utile pour savoir de quel métier relève une ressource). 💡
Depuis un script, `getJobLevel(jobId)` (ou `job:level(jobId)`) renvoie ton niveau dans un métier.

JobID	Métier	JobID	Métier
2	Bûcheron	44	Forgemage
11	Forgeron	48	Sculptemage
13	Sculpteur	60	Façonneur
15	Cordonnier	62	Cordomage
16	Bijoutier	63	Joaillomage
24	Mineur	64	Costumage
26	Alchimiste	65	Bricoleur
27	Tailleur	74	Façomage
28	Paysan	41	Chasseur
36	Pêcheur		

A.3 Interactifs & lieux (ElementTypeld)

Les plus utiles pour un script : Banque 168 , Zaap 16 , Zaapi 106 , Puits 84 , Coffre 85 , Coffre de guildes 388 , HDV ressources 313 / équipements 314 / consommables 315 .

La liste complète des éléments interactifs de la carte (ateliers, zaaps, banques, portes...) est dans le fichier :

📄 Interactives.txt (8.4 KB)

A.4 Monstres (genericId → nom)

À utiliser dans `FORBIDDEN_MONSTERS` (ne jamais attaquer) et `FORCE_MONSTERS` (n'attaquer que ceux-ci). Tu peux mélanger **noms** (entre guillemets) et **genericId** (nombres) :

```
FORBIDDEN_MONSTERS = { "Tofu", "Bouftou royal" } -- par nom
FORCE_MONSTERS      = { 36 }                      -- par id (36 = B
```

Le filtrage par nom ne tient pas compte de la casse. La taille du groupe se règle avec `MIN_MONSTERS` / `MAX_MONSTERS` (§7.4).

Liste complète des monstres (id → nom) :

 [monstres.txt \(62.3 KB\)](#)

A.5 Objets (gid → nom)

À utiliser dans `getInventoryItemCount(gid)`, `useInventoryItem(gid, count)` (ou `useItem`), `deleteItem(gid, count)`, `dropItem(gid, count)` et `AUTO_DELETE`. Exemple courant : ouvrir automatiquement les sacs de ressources récoltées.

Sacs de ressources (les plus utilisés)

gid	Objet	gid	Objet
7941	Sac de blé	7947	Sac de riz
7942	Sac d'orge	7948	Sac de malt
7943	Sac d'avoine	7949	Sac de chanvre
7944	Sac de houblon	16532	Sac de maïs
7945	Sac de lin	16533	Sac de millet

7946	Sac de seigle		
------	---------------	--	--

```
-- Ouvre tous les sacs de blé présents dans l'inventaire  
local n = getInventoryItemCount(7941)  
if n > 0 then useInventoryItem(7941, n) end
```

💡 Plus simple : `openBags()` ouvre d'un coup **tous** les sacs de ressources, et `OPEN_BAGS = true` (§7.4) le fait automatiquement quand ton sac est plein.

La liste **complète des objets** (plus de 19 000 entrées) est trop volumineuse pour être affichée ici : ouvre le fichier et fais `Ctrl+F` pour retrouver un gid.

📄 Objets.txt (527.1 KB)

Dernière mise à jour Sep 26, 2026 at 06:18 PM